电 子 科 技 大 学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目 **资源受限嵌入式环境多模型 AI 推理框架设计**

学科专业	软件工程
学　　号	**202221090110**
作者姓名	王若宇
指导教师	廖勇　　教　授
学　　院	信息与软件工程学院

中国知网　https://www.cnki.net

分类号 TP311.5 密级 公开

UDC 注1 004.41

学 位 论 文

资源受限嵌入式环境多模型 AI 推理框架设计

（题名和副题名）

王若宇

（作者姓名）

指导教师 廖勇 教 授

电子科技大学 成 都

申请学位级别 硕士 学科专业 软件工程

提交论文日期 2025 年 4 月 8 日 论文答辩日期 2025 年 5 月 7 日

学位授予单位和日期 电子科技大学 2025 年 6 月

答辩委员会主席

评阅人

注 1：注明《国际十进分类法 UDC》的类号。

中国知网 https://www.cnki.net

AI Inference Framework for Multi-Model Processing in Resource-Constrained Embedded Environments

A Master Thesis Submitted to

University of Electronic Science and Technology of China

Discipline **Software Engineering**

Student ID **202221090110**

Author **Wang Ruoyu**

Supervisor **Prof. Liao Yong**

School **School of Electronic Science and Engineering**

中国知网 https://www.cnki.net

摘要

近年来，物联网（IoT）作为国家数字化转型的重要推动力量发展迅猛，人工智能（AI）随着软硬件技术的不断成熟也快速普及。二者的融合催生了嵌入式 AI 技术，成为当前研究的重点方向。然而如何在资源受限的嵌入式环境下进行 AI 推理，使得 AI 推理框架与嵌入式实时操作系统（RTOS）密切结合，发挥 RTOS 在混合任务调度中的优势，充分利用有限的嵌入式硬件资源解决 AI 模型部署难题是嵌入式 AI 应用需要解决的问题。

本文基于实验室团队开发的珊瑚（aCoral）嵌入式实时操作系统，设计并实现了一个面向资源受限嵌入式环境的多模型 AI 推理框架，完成了在无 MMU（Memory Management Unit）的嵌入式环境中的虚拟内存管理机制，解决了因内存不足而无法加载 AI 模型完成推理的问题，同时，提出了与实时操作系统高度融合的 AI 任务推理流水线加速模型，进一步提升了嵌入式硬件资源的利用率。本文主要的研究内容包括：

（1）针对无 MMU 的嵌入式环境，采用软件模拟 MMU 功能，并将虚拟内存管理机制集成至 AI 推理框架的模型加载与推理过程中。

（2）对 AI 模型进行结构分析，使得加载推理过程透明可控，解析 AI 任务 DAG 图，并结合实际硬件环境完成 AI 任务的负载划分。

（3）根据 AI 任务 DAG 图和实际硬件条件，完成 AI 任务的线程映射和异构多核调度机制，并提出多核下 AI 推理任务的流水线加速模型。

（4）在 K210 硬件平台中部署珊瑚（aCoral）实时操作系统，基于 Yolox_Nano 模型进行了传统 AI 推理框架和资源受限嵌入式环境多模型 AI 推理框架的对比实验，以验证框架的性能差异。

（5）设计目标检测追踪系统，结合珊瑚（aCoral）实时操作系统任务调度机制以及资源受限嵌入式环境多模型 AI 推理框架，验证混合任务调度下框架的效果。

本文的研究一定程度上解决了嵌入式环境因资源有限难以部署 AI 应用的问题，并为提高嵌入式环境中的硬件资源利用率提出了完整的方案，有助于嵌入式 AI 应用在更广泛的领域部署。

关键词：人工智能，嵌入式实时操作系统，虚拟内存管理，AI 推理框架

中国知网 https://www.cnki.net

ABSTRACT

In recent years, the Internet of Things (IoT) has developed rapidly as a key driving force for national digital transformation, while Artificial Intelligence (AI) has also proliferated with the continuous advancement of software and hardware technologies. The convergence of the two has given rise to embedded AI technology, which has become a major focus of current research.

However, how to perform AI inference in resource-constrained embedded environments, tightly integrate AI inference frameworks with embedded real-time operating systems (RTOS), leverage the advantages of RTOS in hybrid task scheduling, and fully utilize limited embedded hardware resources to address the challenges of AI model deployment, remains a critical issue for embedded AI applications.

This thesis, based on the aCoral embedded real-time operating system developed by the laboratory team, designs and implements a multi-model AI inference framework tailored for resource-constrained embedded environments. It establishes a virtual memory management mechanism for embedded systems without a Memory Management Unit (MMU), solving the problem of insufficient memory that prevents AI models from being loaded for inference. Meanwhile, it proposes an AI task inference pipeline acceleration model that is highly integrated with the RTOS, further enhancing the utilization of embedded hardware resources.

The main contributions of this thesis include:

(1) For embedded environments without an MMU, a software-based MMU simulation is adopted, and the virtual memory management mechanism is integrated into the AI inference framework for model loading and inference processes.

(2) The AI model structure is analyzed to make the loading and inference process transparent and controllable, with AI task DAG (Directed Acyclic Graph) parsing and task load distribution based on the actual hardware environment.

(3) According to the AI task DAG and hardware conditions, AI task thread mapping and heterogeneous multi-core scheduling mechanisms are completed, and a pipeline acceleration model for AI inference tasks under multi-core environments is proposed.

(4) The aCoral RTOS is deployed on the K210 hardware platform, and comparative experiments between the traditional AI inference framework and the proposed multi-

中国知网 https://www.cnki.net

model AI inference framework for resource-constrained embedded environments are conducted based on the Yolox_Nano model to verify performance differences.

(5) A target detection and tracking system is designed, combining the aCoral RTOS task scheduling mechanism and the proposed inference framework, to validate the effectiveness of hybrid task scheduling.

The research presented in this thesis addresses the challenge of AI application deployment in resource-constrained embedded environments to some extent and proposes a complete solution to improve hardware resource utilizati on, facilitating broader deployment of embedded AI applications.

Keywords: Artificial Intelligence, Embedded Real-Time Operating System, Virtual Memory Management, AI Inference Framework

中国知网 https://www.cnki.net

目　录

第一章 绪论.....1
1.1 研究背景与意义.....1
1.2 国内外研究现状.....2
1.2.1 发展历史.....2
1.2.2 主流技术概述.....3
1.2.3 存在的问题.....5
1.3 研究内容与贡献.....6
1.4 论文组织结构.....7
第二章 相关理论技术与推理框架总体设计.....8
2.1 嵌入式环境 AI 推理框架.....8
2.1.1 神经网络模型编译与优化.....8
2.1.2 神经网络模型运行时环境.....9
2.2 嵌入式虚拟内存管理技术.....11
2.2.1 虚拟内存映射.....12
2.2.2 虚拟内存页式内存管理设计.....12
2.2.3 虚拟内存共享与保护.....13
2.3 嵌入式 AI 应用开发平台.....14
2.4 资源受限嵌入式环境多模型 AI 推理框架总体设计.....16
2.5 本章小结.....17
第三章 AI 模型虚拟内存管理.....18
3.1 AI 模型嵌入式环境部署分析.....18
3.2 虚拟内存管理器设计.....19
3.3 编程接口实现.....21
3.3.1 虚拟内存初始化接口.....21
3.3.2 虚拟内存申请接口.....23
3.3.3 虚拟内存访问接口.....24
3.3.4 虚拟内存释放接口.....26
3.3.5 虚拟内存共享接口.....27
3.4 底层硬件驱动实现.....28
3.5 本章小结.....31
第四章 AI 模型结构分析与负载划分.....32
4.1 AI 模型加载与分析.....32
4.1.1 AI 模型头部结构.....34
4.1.2 AI 模型运算模块结构.....35

中国知网 https://www.cnki.net

4.1.3 AI 模型加载分析.....39
4.2 AI 模型算子与张量结构分析.....42
4.3 AI 模型虚拟内存映射.....44
4.4 AI 任务负载划分.....45
4.5 本章小结.....46
第五章 AI 任务异构多核调度与推理加速.....47
5.1 AI 模型推理任务线程映射.....47
5.2 AI 模型推理任务异构多核调度.....50
5.2.1 珊瑚(aCoral)实时操作系统事件组扩展.....50
5.2.2 AI 模型推理任务线程调度控制.....52
5.3 AI 模型推理任务多核流水线加速方法.....54
5.4 本章小结.....56
第六章 多模型 AI 推理框架的实现与测试.....57
6.1 资源受限嵌入式环境 AI 框架实现流程.....57
6.2 资源受限嵌入式环境多模型 AI 推理框架对比测试.....58
6.3 基于目标检测追踪的实例验证.....61
6.3.1 系统设计与实现流程.....61
6.3.2 K210 核心负载分析.....64
6.4 本章小结.....67
第七章 全文总结与展望.....68
7.1 全文总结.....68
7.2 后续工作与展望.....69
参考文献.....70

中国知网 https://www.cnki.net

第一章 绪论

1.1 研究背景与意义

近年来，物联网（IoT）作为推动国家数字化转型的关键组成部分，发展势头迅猛。随着物联网终端硬件设备成本的显著下降及各种无线通信技术的广泛应用，物联网终端设备的数量持续攀升。这一变化不仅促进了智能家居、智能城市等领域的快速发展，也推动了工业自动化等高端应用的革新，进一步巩固了物联网作为未来技术发展主流的地位。同时另一热门话题，人工智能（Artificial Intelligence，AI），也由于大模型（Large Model）以及高性能图形处理器（Graphic Processing Unit，GPU）、神经网络处理器（Neural network Processing Unit，NPU）等软、硬件技术的成熟，得到了飞速发展。所以二者的融合应用，嵌入式 AI 技术也成为重点研究内容，并且在智慧城市、自动驾驶、工业物联网、智能医疗等领域有越来越广阔的应用前景。

尽管深度学习已经成为人工智能领域中最为重要的技术，但由于神经网络模型本身特有的大量计算和巨大的模型参数尺寸，使得神经网络模型落地应用部署的硬件要求非常高。因此，在资源受限的物联网嵌入式终端设备上运行深度学习应用有着很多的问题需要解决。

由于受到硬件资源的限制，神经网络模型的训练过程无法在物联网终端设备完成，所以人工智能与物联网的融合工作主要集中在模型的推理过程。现有的神经网络模型终端推理分为云端推理和边缘本地推理两种方式。前一种方式中，边缘设备通过与资源丰富的云端无线连接，能够完成复杂场景下的高吞吐量运算，但是这种方案也带来诸多问题。首先是用户的数据隐私安全问题。使用云端进行模型推理时，用户往往需要向云端暴露隐私数据。其次，云端推理需要与终端保持稳定的常态连接，而在某些特殊区域中无法保证连接的稳定性与实时性。此外，在类似于航天航空、铁路交通等领域的实时应用需要低延迟的连接，而远距离的网络延迟受软硬件，环境等诸多因素的影响，波动较大。最后，无线连接能耗比相对较高，远距离传输需要耗费较多的能量。所有这些问题(隐私、延迟、连接性以及昂贵的无线连接)都可以通过在边缘端实施推理计算来解决[1]。

但在边缘本地推理中同样存在诸多问题需要解决。其中，最主要的问题在于终端设备资源受限的硬件特性。无论是存储资源还是计算资源，嵌入式设备的硬件资源远不如服务器或桌面系统。它们的计算能力通常仅能支持特定任务的单模型推理，在需要推理多个模型时容易导致性能瓶颈。目前主流的解决方案之一是对模型

中国知网 https://www.cnki.net

进行轻量化处理。虽然模型压缩（如剪枝、量化）可以降低模型的资源占用，但在多模型推理场景下，压缩模型仍然不足以彻底解决资源争用问题。此外，部分应用需要不同精度的模型，这使得模型轻量化技术的效果可能有所差异。最后，当前主流的嵌入式 AI 推理框架（如 TensorFlow Lite、ONNX Runtime、TVM）主要针对单一模型优化，多模型协同推理的研究和支持较为有限。在资源受限环境中，如何与嵌入式实时操作系统（RTOS）进行高层次的融合，借助 RTOS 高效的调度和资源共享机制完成多模型协同推理，这些 AI 推理框架都还欠缺相应的支持。

本文的研究，有望构建一种资源受限嵌入式环境多模型 AI 推理框架，一定程度上解决嵌入式终端设备资源受限的问题，完成框架与珊瑚（aCoral）嵌入式实时操作系统的融合工作，解决现有嵌入式 AI 推理框架多模型协同推理机制缺乏等问题。

1.2 国内外研究现状

1.2.1 发展历史

嵌入式 AI 技术的发展历史可以追溯到嵌入式系统和人工智能独立发展的早期阶段。在 20 世纪末，嵌入式系统主要应用于消费电子和工业控制等特定场景，重点在于低功耗、高可靠性。而人工智能则专注于研究层面的探索，如专家系统和基本的机器学习算法[2]。然而，当时由于嵌入式硬件计算能力有限，以及 AI 算法的高复杂度，嵌入式 AI 技术仅仅停留在概念层面。

进入 21 世纪，由于深度学习技术的蓬勃发展与硬件性能的飞跃提升，人工智能迈入前所未有的高速发展阶段。尤其是 2012 年 AlexNet 在图像分类任务上的成功，使得神经网络成为人工智能技术的核心。这一突破推动了 AI 算法向更高效、更实用的方向发展，同时随着嵌入式硬件（如 ARM 架构处理器）的性能显著提升，简单的 AI 任务开始被部署在嵌入式设备中，如早期的语音助手和智能家居设备。

21 世纪 10 年代中期，嵌入式 AI 迎来了快速发展。这一阶段由于神经网络计算专用加速硬件的出现，例如 NVIDIA 的 Jetson 平台和谷歌的 TPU，以及边缘计算概念的兴起，嵌入式 AI 开始向边缘计算的方向发展[3]。与此同时，工具链的不断完善（如 TensorFlow Lite 和 ONNX）使得开发者可以更方便地将神经网络模型部署到嵌入式设备。软、硬件技术的提升使得神经网络运算可以在本地完成，从而降低了对云端的依赖，这种转变极大地推动了嵌入式 AI 技术的发展。

进入 2020 年后，嵌入式 AI 技术已广泛渗透到各个领域，成为智能物联网发展的核心驱动力。硬件方面，专用芯片（如寒武纪、华为昇腾）和轻量化推理框架

中国知网 https://www.cnki.net

不断涌现，为复杂 AI 任务的高效处理提供支持。软件算法方面，通过模型剪枝、量化和知识蒸馏等技术，复杂的深度学习模型被大幅优化以适应资源有限的嵌入式环境。如今，嵌入式 AI 已被应用于智慧医疗、智能制造、农业监控等领域，并在实时性、可靠性和隐私保护方面展现出巨大优势。

尽管取得了显著进展，嵌入式 AI 应用仍然面临硬件资源有限、模型优化难度高以及隐私与安全性难以保证的问题。随着技术的持续创新，嵌入式 AI 应用有望在未来继续推动全球数字化进程，为社会带来更深远的变革。

1.2.2 主流技术概述

在嵌入式 AI 领域，开发人员可选择多种嵌入式 AI 推理框架和性能各异的硬件部署平台，比如国内百度的 Paddle Lite 框架和国外的 STM32Cube.AI 和 TensorFlow Lite 等框架。这些框架各自有着独特的模型优化策略和应用部署方案，在国内外市场都受到广泛关注和认可，为嵌入式端深度学习模型的部署奠定了基础，加速了边缘设备智能化的进程[4]。

Paddle Lite 是百度基于自研深度学习平台 PaddlePaddle 而推出的一款嵌入式端轻量级推理框架。它的目的在于为手机和物联网终端设备提供深度学习模型的轻量级部署方案，集成了跨平台的硬件支持并且做到对主流框架深度学习模型的兼容。图 1-1 为 Paddle Lite 框架的架构图。

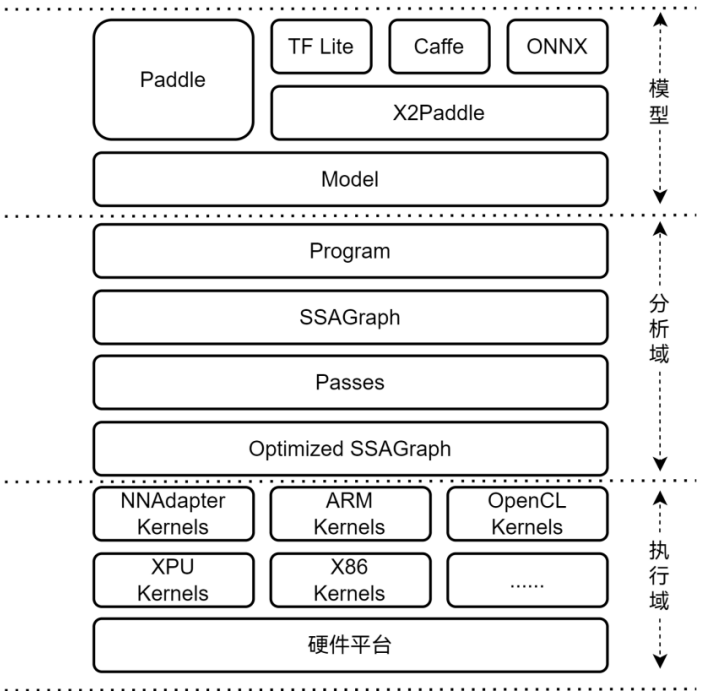


图 1-1 Paddle Lite 架构图

中国知网 https://www.cnki.net

Paddle Lite 框架呈现三层结构，各层分别承担不同的功能。模型层主要完成 Paddle Lite 框架与主流神经网络框架的兼容工作，将主流框架的神经网络模型进行格式转换，包括 Paddle、TF Lite、Caffe、ONNX 等；分析域层负责优化模型的计算图结构，通过优化和压缩 Program、Passes 以及 SSAGraph 等数据来提升神经网络模型的性能；执行域层则与底层硬件平台密切相关，不仅对 ARM、X86 平台等硬件平台提供驱动支持，还兼容了 OpenCL 等异构并行编程框架。Paddle Lite 框架在加速和优化方面表现优异，采用了量化、子图融合、Kernel 优选等多种技术手段，使得优化后的模型更加轻量化，占用更少的资源，同时执行效率显著提升[5]。

STM32Cube.AI 框架是 STMicroelectronics 公司开发并提供技术支持的一款嵌入式端 AI 应用开发工具。开发者在 PC 端训练完成的神经网络模型可以直接导入框架，然后框架将其转换为可以在 STM32 微控制器上直接部署的代码[6]。图 1-2 为该框架的工作流程图。

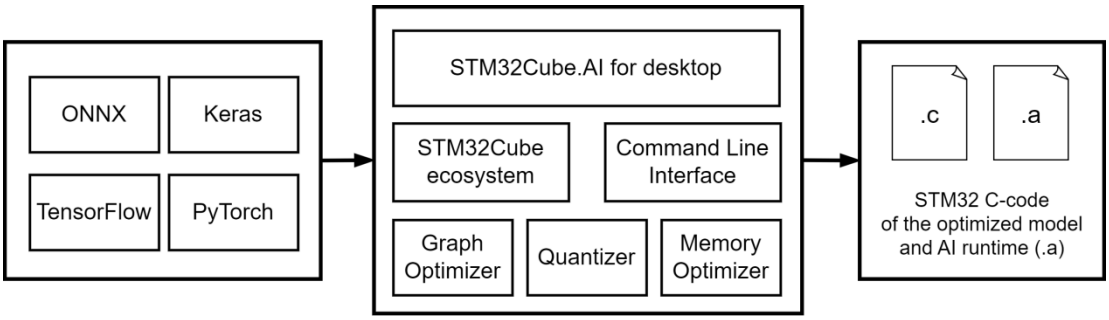


图 1-2 STM32Cube.AI 工作流程

STM32Cube.AI 的工作流程分为输入模型、模型优化和输出模型三步[7]。STM32Cube.AI 与 ST 公司开发的 STM32CubeMX 开发平台高度契合，极大地简化了嵌入式 AI 应用的开发流程。然而这种与 STM32 硬件平台高度兼容的框架设计虽然能提高嵌入式 AI 应用的开发效率，但也导致了开发者学习门槛变高、嵌入式 AI 应用可移植性差等问题。

TensorFlow Lite 是 Google 公司开发的一款嵌入式端神经网络模型推理框架，用于在手机、微控制器和其他嵌入式端设备上进行嵌入式 AI 应用开发，对 Android、IOS 和嵌入式 Linux 等平台都有良好的软件支持 [14]。图 1-3 为 TensorFlow Lite 工作流程。

TensorFlow Lite 框架和 TensorFlow 框架高度绑定，开发者可在 PC 端完成神经网络模型的训练过程，接着借助 TensorFlow Lite 框架将模型编译成嵌入式端支持的模型格式，然后在嵌入式端 TensorFlow Lite 解析器读取该模型并进行模型推

中国知网 https://www.cnki.net

理，最终开发者根据具体需求完成嵌入式 AI 应用开发。TensorFlow Lite 框架的优势体现在它不仅高度兼容主流框架 TensorFlow，并且还对模型进行了轻量化处理，使得模型更适合嵌入式环境，并且还基于神经网络计算加速硬件对神经网络模型的推理过程进行了计算加速处理[15]。但是这款框架的缺点也十分明显，TensorFlow Lite 要求嵌入式环境必须支持一定的软件环境，比如 Android、IOS 和嵌入式 Linux 等[8]，这导致了 TensorFlow Lite 的适用性降低，并且增加了嵌入式 AI 应用开发的软、硬件成本。

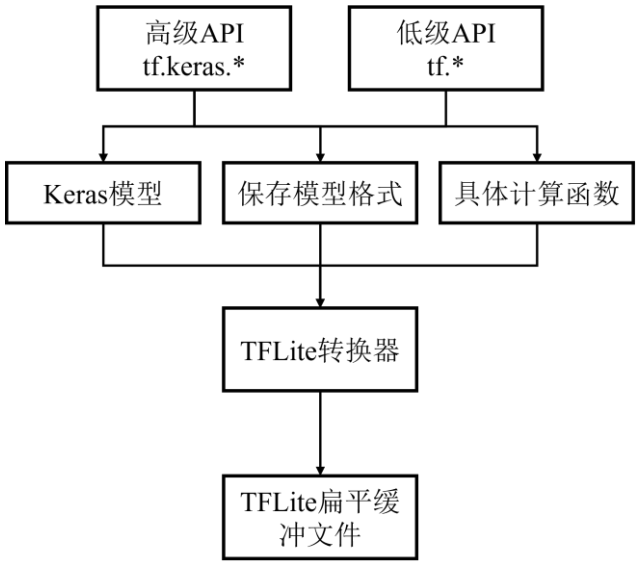


图 1-3 TensorFlow Lite 工作流程

1.2.3 存在的问题

现有嵌入式 AI 推理框架发展已经比较完善，在嵌入式端的神经网络模型优化以及推理流程都有着各自的优点，但是这些框架或多或少的对嵌入式端软件和硬件环境都有较高的要求[9]，比如需要特定的某款微控制器、需要使用指定的软件开发平台、需要底层特定操作系统支持等，这些要求在资源受限的嵌入式环境中往往难以满足。

另外通过学习现有嵌入式 AI 推理框架，不难发现这些框架在嵌入式端部署神经网络模型应用时的重心都放在模型本身优化上，试图更改模型本身来适应嵌入式环境中资源不足的问题，通过尽可能减少神经网络模型的参数量和计算量来提升在嵌入式平台上的推理效率。比如针对模型权重数据的量化处理、模型的子图融合、Kernel 优选和知识蒸馏等手段。但是在一些极端情况下，这些模型的优化处理

中国知网 https://www.cnki.net

仍不能满足实际的嵌入式环境硬件需求，导致了嵌入式 AI 应用开发难以完成的问题[10]。

除此之外，这些框架还存在一些共同的问题。首先是单一地针对模型本身进行优化，而忽略了框架对实际硬件资源的合理分配与调度的支持，导致了在多模型推理场景下的硬件资源争用问题，并且有限的资源也不能得到充分利用。在实际的嵌入式 AI 应用开发中，这些框架不能很好的结合具体的硬件环境来针对神经网络模型的部署以及推理过程进行优化处理，比如硬件内存有限的模型部署策略、多核 CPU 下的模型推理负载划分、NPU 异构加速硬件的高效分配调度等。其次是这些框架几乎没有涉及与嵌入式实时操作系统（RTOS）的高层次融合，无法结合 RTOS 在嵌入式端的高效调度策略和资源共享机制，使得目前嵌入式端的 AI 应用仍以类似于单板裸机程序的方式实现，无法保证在某些特别领域中系统的实时性要求，降低了系统的稳定性与安全性。

1.3 研究内容与贡献

为解决上述问题，本文提出了一种资源受限嵌入式环境多模型 AI 推理框架的设计方案，从 AI 模型虚拟内存管理、AI 模型结构分析与负载划分、AI 任务异构多核调度与推理加速以及目标检测追踪系统应用四个方面展开研究，具体研究内容包括：

（1）AI 模型虚拟内存管理：建立嵌入式 AI 模型虚拟内存管理机制，实现资源受限嵌入式平台的 AI 推理运算；设计 AI 模型权重保护方法，避免多模型运行时模型间的数据权重破坏，提供模型私有数据安全保护机制；研究虚拟内存共享问题，提供多模型 AI 推理时数据共享与隔离机制；应用虚拟内存管理技术解决了在嵌入式端因运行内存不足而无法加载神经网络模型进行推理的问题。

（2）AI 模型结构分析与负载划分：设计 AI 任务细粒度结构模型，建立 AI 任务 DAG 结构，设计支持 AI 任务高效执行的线程控制结构及队列；根据硬件条件完成 AI 任务负载划分，并实现相关管理机制确保 AI 任务的正确执行；通过对 AI 模型的结构分析以及负载划分，解决了主流嵌入式端 AI 推理框架推理流程不透明的问题，保证了 AI 推理任务的实时可控。

（3）AI 任务异构多核调度与推理加速：研究异构多核环境下 AI 任务并行调度算法，结合嵌入式实时操作系统任务管理机制保证 AI 任务的实时可控，构建支持多个 AI 推理任务加载与并发执行的多模型调度机制；类比 CPU 指令流水线模型提出 AI 任务推理流水线加速模型，并完成相应的 AI 子任务划分管理机制，最终

中国知网 https://www.cnki.net

实现 AI 任务推理加速的目标；结合实时操作系统的多核调度管理，解决了嵌入式端 AI 推理框架与实时操作系统融合度低的问题，提高了硬件资源的利用率。

（4）框架性能对比实验：在 K210 硬件平台中部署珊瑚（aCoral）实时操作系统，基于 Yolox_Nano 模型完成传统 AI 推理框架和资源受限嵌入式环境多模型 AI 推理框架的对比实验。

（5）目标检测追踪系统：基于珊瑚（aCoral）操作系统，结合现有 AI 运行支撑库，实现多核情景下 AI 模型推理加速机制，构建支持 AI 推理任务与嵌入式实时操作系统混合实时任务调度机制深度协同的实时可控推理框架，在智能控制、目标识别等典型场景进行实验，并将框架与现有框架进行对比测试。

本文研究内容属于嵌入式 AI 技术与实时操作系统技术相结合的领域，针对有实时性和 AI 应用需求的资源受限嵌入式系统，构建资源受限嵌入式环境多模型 AI 推理框架，以支撑实时嵌入式 AI 应用开发。

1.4 论文组织结构

本文主要分为以下七章：

第一章主要介绍资源受限嵌入式环境多模型 AI 推理框架的研究背景与意义，并通过介绍现有的几种典型的嵌入式 AI 运行框架技术，阐述它们现阶段共有的一些问题，最终引出本文将要研究的内容、目标与本文的贡献。

第二章介绍在框架中的设计研发过程中使用的技术要点与理论细节，包括嵌入式环境 AI 推理框架、嵌入式虚拟内存管理技术以及资源受限嵌入式环境多模型 AI 推理框架总体设计等内容。

第三章主要介绍 AI 模型虚拟内存管理的设计与实现，包含了神经网络模型虚拟内存管理技术的具体实现方式以及相应的软硬件编程接口。

第四章主要介绍框架中的 AI 模型结构分析与负载划分内容，包含了神经网络模型的结构解析、DAG 模型映射和 AI 任务负载划分等内容。

第五章主要介绍本文研究框架中 AI 任务异构多核调度策略与推理加速模型。

第六章主要介绍为验证资源受限嵌入式环境多模型 AI 推理框架而设计的与现有框架的对比实验，并通过一个目标检测追踪系统来体现框架与珊瑚（aCoral）实时操作系统的高度融合。

第七章主要介绍工作内容的完成情况、存在的不足以及未来的发展预期等。

中国知网 https://www.cnki.net

第二章 相关理论技术与推理框架总体设计

本文所研究的资源受限嵌入式环境 AI 推理框架，主要涉及到的理论与技术包括嵌入式环境AI推理技术、嵌入式虚拟内存管理技术，以及本文使用的K210 RISC-V 架构多核 SOC 硬件平台。本章将对这些理论与技术进行介绍并提出资源受限嵌入式环境 AI 推理框架的总体设计。

2.1 嵌入式环境 AI 推理框架

2.1.1 神经网络模型编译与优化

在嵌入式环境 AI 推理框架中，神经网络编译器是不可或缺的组成部分之一。神经网络编译器的目标是将在服务器或桌面系统上部署的神经网络模型进行特殊处理，将之转换为适合特定嵌入式环境（比如微控制器、NPU、TPU、FPGA 等）的轻量模型[11]。在嵌入式环境中，由于设备的计算和存储能力都受到限制，传统的神经网络模型往往难以直接部署，所以需要借助神经网络编译器对神经网络模型进行优化处理，包括但不限于量化、剪枝、算子融合等手段，以此降低神经网络模型的存储体积和计算量来适应嵌入式系统的硬件限制。同时神经网络编译器还能结合实际的硬件条件，对模型的参数、算子布局等细节进行优化处理，使模型在特定硬件（比如 NPU、TPU 等）上的运行性能得到提升。

本文使用的嵌入式环境 AI 推理框架是 nncase。作为一款成熟的框架，nncase 的神经网络编译器不仅兼容多种主流框架的神经网络模型，而且还在模型转换中针对嵌入式环境对模型的数据存储方式以及网络结构提出了不少优化手段[12]，其神经网络编译器架构如图 2-1 所示。

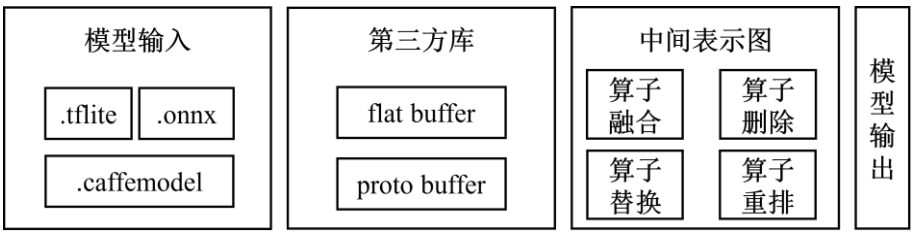


图 2-1 nncase 神经网络编译器架构图

在 nncase 中，模型的编译过程分为四个步骤。第一步是模型的输入，传统的深度学习框架训练得到的模型文件格式各自不同，nncase 的神经网络编译器针对

中国知网 https://www.cnki.net

这些不同的文件格式（比如 tflite、onnx、caffemodel）都做了相应的编译支持，能够从这些不同的文件格式中解析出神经网络模型的网络结构和权重参数，做到与主流框架的良好兼容。第二步是模型的转换过程，这一步骤中，神经网络编译器根据主流框架训练模型时使用的第三方库对模型进行转换，将不同输入格式的模型文件转划为一种中间表示图, 以便完成后续的模型优化处理工作。第三步是模型的优化，中间表示图是模型的计算图，计算图是一个有向无环图 (Directed Acyclic Graph， DAG)。计算图中的节点和边分别代表算子和算子的数据依赖关系[19]，神经网络编译器根据计算图对模型的网络结构和计算顺序做出优化以适应嵌入式运行环境，并且根据编译参数对模型进行权重参数的量化处理来减少模型的运行内存需求。第四步是完成模型的输出，神经网络编译器完成以上三个步骤后，需要将计算图表示的模型转换为在嵌入式环境部署的特定格式的模型文件，这种格式的文件内存分布紧凑并且占用空间远远小于处理前的模型文件，适合在嵌入式环境中进行推理并部署相应的嵌入式 AI 应用。

2.1.2 神经网络模型运行时环境

神经网络模型运行时环境（即 AI 运行时环境）是嵌入式 AI 推理框架的核心部分。nncase 推理框架分为模型编译和模型运行时环境两部分，模型编译负责完成模型的转换与优化过程，模型运行时环境负责在嵌入式环境中为神经网络模型的推理提供软件支持[13]。nncase 的模型运行时环境架构图如图 2-2 所示。

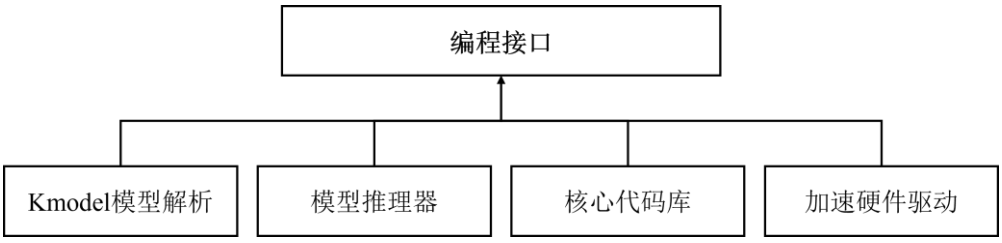


图 2-2 nncase 模型运行时环境架构图

nncase 的模型运行时环境向上为用户提供编程接口，向下分为四个模块实现运行时环境的不同功能。

第一个模块负责模型的加载与解析功能。nncase 推理框架的神经网络编译器将模型编译完成后得到 Kmodel 模型文件，该模块将从 Flash 中读取模型文件并根据模型文件的头部信息完成模型运行时对象的初始化，以便后续完成推理。第二个模块为模型推理器部分，完成模型的加载解析和输入数据的处理后，推理器将根据模型文件中的算子计算顺序以及权重数据来调用核心代码库中的相应代码进行模

中国知网 https://www.cnki.net

型的推理，并将推理结果保存到指定的内存地址。第三个模块是模型运行时环境的核心代码库，主要由各种算子的代码实现组成，用户可自行对代码库进行裁剪、改进，能灵活地适应不同的用户需求。第四个模块为 KPU 神经网络加速器硬件提供驱动支持，nncase 推理框架实现了适配 KPU 进行硬件加速的专用算子，这一模块主要为这些算子提供底层的硬件驱动。

nncase 的模型运行时环境连接了嵌入式 AI 应用开发与 AI 推理框架底层运行逻辑，它的运行流程图如图 2-3 所示。

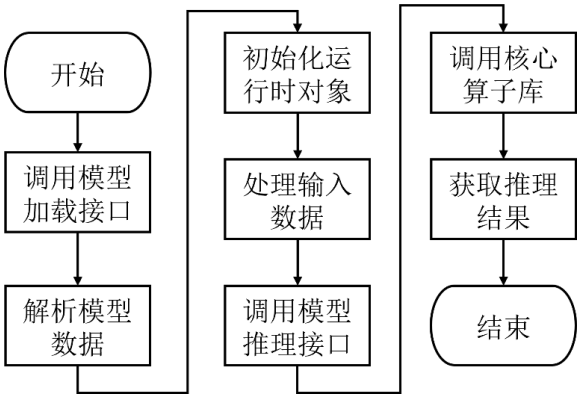


图 2-3 模型运行时环境流程图

（1）在开发嵌入式 AI 应用时，开发人员通过调用模型加载编程接口并提供模型文件存储地址将编译好的 Kmodel 文件加载到嵌入式模型运行时环境中。

（2）模型加载完毕后，框架将从模型文件中读取模型的结构信息（包含模型模块数、算子数、模型体积等数据），然后初始化模型推理的运行时对象。

（3）模型初始化完成后，框架通过嵌入式终端的输入设备获取到输入数据并将数据存储到内存当中，框架根据模型运行时对象，完成输入数据的张量化操作，并根据模型的结构信息分配出推理结果的存储空间。

（4）输入数据处理完毕后，框架将调用模型推理接口进行模型推理，这一过程将根据模型文件中存储的算子信息按序执行。

（5）在进行模型推理时，框架会根据算子的计算顺序调用核心代码库中的算子实现，并将推理结果存储在之前分配的内存地址中。

（6）嵌入式 AI 应用开发人员可调用模型推理结果获取接口从模型运行时环境中获取推理结果，对推理结果进行处理后便可以通过嵌入式输出设备向用户展示模型推理结果，从而实现嵌入式 AI 应用，比如图像识别、实时翻译等。

中国知网 https://www.cnki.net

2.2 嵌入式虚拟内存管理技术

在物联网高速发展的时代，嵌入式设备已经彻底改变了我们的日常生活，比如智能家居、智能可穿戴设备、汽车电子、移动支付等日常应用，为我们的衣食住行带来了不少便利。在这些产品的背后，微控制器硬件技术的不断发展充当了重要角色[14]。从最早仅支持频率在几 Khz 的微小 4 位系统到如今微处理器频率高达数百 MHz、数百 GHz 的复杂 32 位、64 位系统，处理器计算频率的稳步提高使得在小型嵌入式设备上运行复杂且计算密集型的应用程序成为可能，而这些应用程序以前只能在服务器或桌面系统上运行。从内存的角度来看，在嵌入式设备上运行更大、数据更密集的应用程序也有类似的需求。然而，对于低端嵌入式处理器和基于微控制器的系统来说，大内存地址空间特别是虚拟内存的支持仍然缺乏。

虚拟内存是一种内存管理方案，它以理想的 RAM 访问速度向软件层提供了一个非常大的连续内存地址空间（与二级存储一样大）[15]。虚拟内存除了提供比 RAM 内存更大的地址空间之外，还支持诸如多任务处理（例如，多个进程访问不相交的大地址空间）和隐私保护（例如，禁止一个进程访问另一个进程的数据）之类的功能。在大多数情况下，虚拟内存是作为操作系统特性之一实现的。

操作系统中虚拟内存的实现很大程度上依赖于底层处理器体系结构，特别是内存管理单元（MMU，Memory Management Unit）。MMU 是一个与核心处理器相关联的硬件单元，它提供对虚拟地址的寻址操作。然而，与高端处理器不同的是，微控制器和大量低端嵌入式处理器并不具备 MMU 硬件。表 2-1 列出了一些无 MMU 的嵌入式处理器[7,8,9,10,11,12]。因此，在大多数情况下，此类处理器下运行的实时操作系统是不支持虚拟内存管理的。所以，拥有处理能力并不一定意味着能够运行数据密集型程序，除非处理了进程地址空间问题，特别是虚拟内存管理技术的支持。

表 2-1 无 MMU 的嵌入式处理器

处理器名称	生产厂家	处理器频率
STM32FXXX	STMicroelectronics	48-216(MHz)
Raspberry Pi RP2040	Raspberry Pi Foundation	133(MHz)
Texas Instruments MSP430	Texas Instruments	16(MHz)
Loongson 1C	Loongson Technology	400(MHz)
GD32F103	GigaDevice (兆易创新)	72(MHz)
Xiaomi Lantian	Xiaomi (小米)	300(MHz)

中国知网 https://www.cnki.net

2.2.1 虚拟内存映射

在虚拟内存管理中，用户程序感知到的地址空间是一块连续的虚拟地址空间。指令访问、数据读写等操作都是访问的虚拟地址，而指令和数据本身是存储在实际的物理内存中，所以虚拟地址和物理地址之间必然存在一种对应关系，这种关系被称作内存映射[16]。内存映射是指将虚拟地址转换为物理地址，这种映射功能是虚拟内存管理中最基本的功能，其他的虚拟内存管理机制都要依托于内存映射来实现。在具备 MMU 硬件的处理器环境中，内存映射的功能并不需要开发人员以软件的形式实现，MMU 能自行的将用户程序访问的虚拟地址转换为对应的物理地址，并且在转换的过程中完成指令和数据的读取写回操作[17]，在这个过程中 MMU、CPU、内存之间的简易关系如图 2-4 所示。

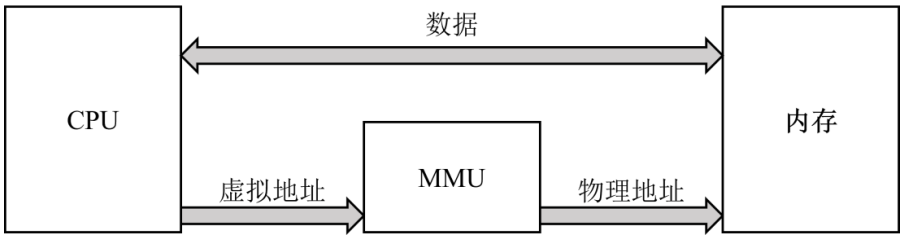


图 2-4 MMU、CPU、内存简易关系图

内存映射中，虚拟内存地址和物理内存地址是一种多对多的映射关系，而且在页式内存管理中，地址转换通常是以页为最小单位（此外还有段式管理和段页式管理，这二者的映射最小单位有所差异），所以在页式虚拟内存管理中，虚拟内存和物理内存的起始地址都是页大小进行字节对齐的。由于 MMU 的存在，开发人员不用太过关心这个过程中的映射方式，只用设置好 MMU 的相关参数就可以使用虚拟内存管理机制，但在缺少 MMU 的硬件环境中，这种机制是不存在的，用户能使用的地址空间只有实际的物理地址空间，尽管处理器的计算性能达到了要求，但是很多数据密集型程序往往也无法直接运行。

2.2.2 虚拟内存页式内存管理设计

虚拟内存中的内存管理有多种实现方式，包括页式内存管理、段式管理和段页式管理[18]，这三者在具体实现细节上各有优劣，此处仅简单介绍页式内存管理的相关技术。

在最基本的虚拟内存的页式内存管理中包含虚页、页框、页表三种元素，虚拟内存会被划分为固定大小的内存块作为虚页，物理内存会被划分为同样大小的内

中国知网 https://www.cnki.net

存块作为页框，同时使用页表来记录虚页和页框之间的映射关系，这三者的关系保证了高效的内存管理和内存虚拟化的可行性。在页式内存管理的机制中，MMU 负责维护页表的数据结构，完成虚拟地址到物理地址的转换过程，同时在遇到虚拟页未命中触发缺页中断的情况下，MMU 还通过合适的页面置换算法（比如 LRU、FIFO 算法）寻找到适合换出的物理页框并根据页面的访问属性进行页面置换，这些页面访问属性包括但不限于页面是否可写、是否采用 Cache、是否全局页等。MMU 通过高效的页面置换算法以及这些页面访问属性可以有效的保证内存访问安全，提高系统的可靠性[19]。在基于页表的虚拟内存管理方式下的 MMU 工作过程如图 2-5 所示。

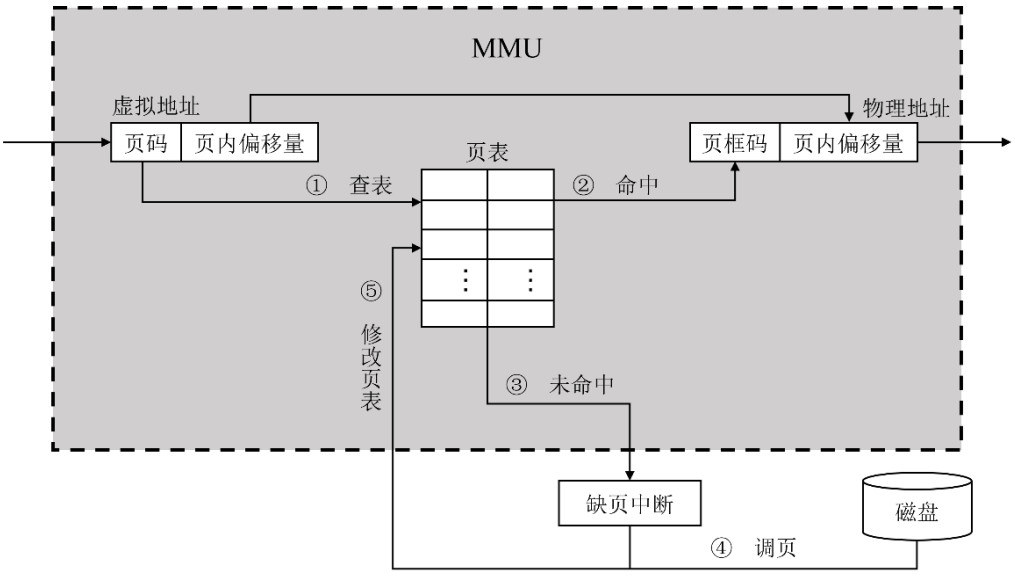


图 2-5 MMU 工作流程

2.2.3 虚拟内存共享与保护

虚拟内存共享是指在虚拟内存管理下，存在多个程序可能需要访问同一块内存数据的情况[20]。在虚拟内存管理中，由于程序访问数据都是通过访问虚拟地址实现的，而虚拟地址会被 MMU 转换为实际的物理地址从而获取程序预期的数据内容，那么只要将不同程序访问的虚拟地址页在页表中指向同一个页框即可实现内存数据共享，这样当多个程序运行时访问这些虚拟地址时实际上是在对同一块物理内存进行访问。多个程序共享同一块物理内存时的关系如图 2-6 所示。

由虚拟内存页式管理的设计理念可知，页表是从虚拟地址转换为物理地址的入口，所以要实现物理内存的保护措施就应该针对页表中这一转换过程做出相应

中国知网 https://www.cnki.net

的设置。常见方式之一是在页框码中设置一些控制字段，用于表明该页允许何种操作，从而禁止非法操作破坏内存安全。图 2-7 所示是一种控制字段的设置方式[21]。

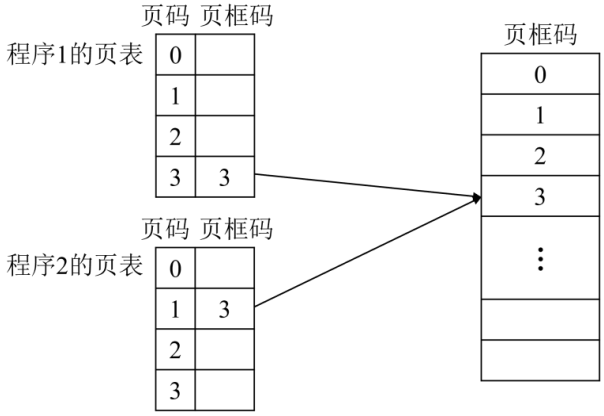


图 2-6 内存共享

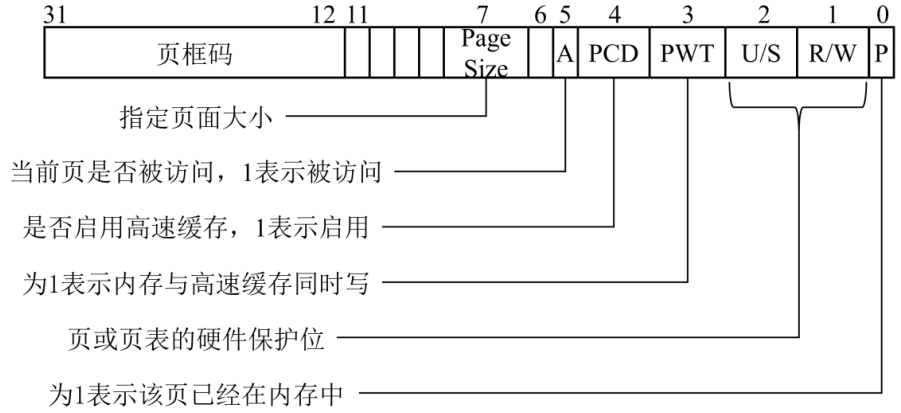


图 2-7 页框码控制字段

当程序对一个页面进行读写等操作时，虚拟内存管理的相关模块则会根据页框码中设置的控制字段对程序操作进行合法性判断，当程序发出非法操作时则会引起一次操作系统的非法访问中断拒绝此次操作，从而做到物理内存中的数据不被恶意破坏，提高系统的健壮性。

2.3 嵌入式 AI 应用开发平台

由于神经网络模型在边缘设备完成推理计算有许多特别的硬件要求，而且传统的嵌入式设备往往不能满足其需求，所以使用专门的嵌入式 AI 硬件平台进行嵌入式 AI 应用开发是十分必要的[22]。嵌入式 AI 硬件平台是一种专门设计用于运行人工智能算法的硬件设备，其目的是在资源受限的嵌入式环境中实现高效的 AI 推

中国知网 https://www.cnki.net

理和计算。这些硬件平台通常都具有高效的计算能力，能对神经网络计算进行硬件加速处理，同时还具备低功耗、小型化、集成化等特性，满足嵌入式环境特殊要求。

本文选择了 Kendryte 勘智公司所研发的 K210 系统级芯片（SOC），该芯片使用台积电 28nm 的先进制程，包含两颗 RISC-V 架构的 64 位 CPU。同时，该芯片还具备丰富的外设单元和神经网络运算加速器供机器视觉、机器听觉等智能应用使用，在实际的嵌入式 AI 应用中，这款芯片拥有良好的功耗表现，也表现出较好的稳定性与可靠性[23]。K210 的硬件架构如图 2-8 所示。

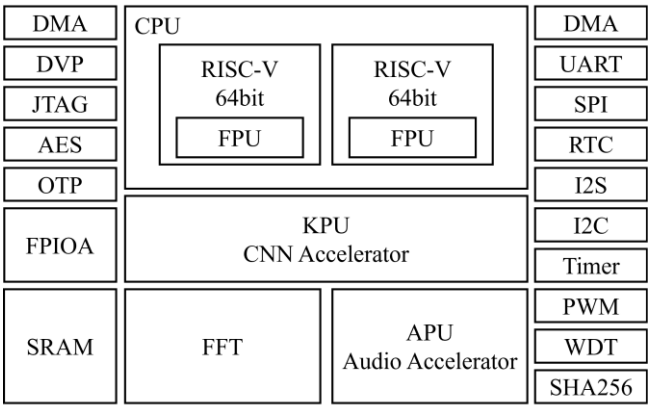


图 2-8 K210 架构图

除了良好的硬件适配外，这款芯片在软件方面也兼容了 nncase 推理框架，从模型的编译优化到运行时部署，该芯片都能借助 nncase 推理框架完成嵌入式 AI 应用的实际开发。整体的开发流程如图 2-9 所示。

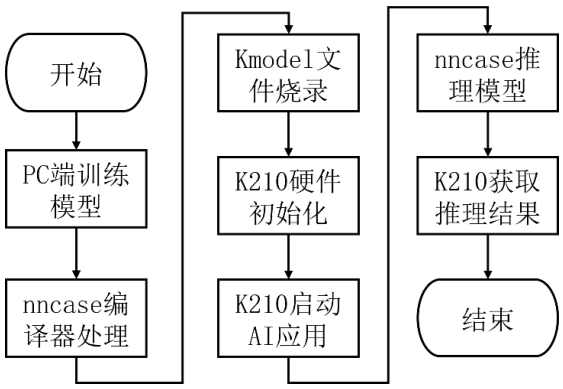


图 2-9 K210 AI 应用开发流程

中国知网 https://www.cnki.net

2.4 资源受限嵌入式环境多模型 AI 推理框架总体设计

本文所研究的资源受限嵌入式环境多模型 AI 推理框架主要是为了解决嵌入式环境中存储空间和计算能力有限而导致嵌入式 AI 应用开发困难重重的问题。针对这两个主要问题，框架主要由三大模块组成：AI 模型虚拟内存管理、AI 模型结构分析、AI 任务异构多核调度与推理加速。框架整体以一个完整的系统呈现，系统的整体架构如图 2-10 所示，其中虚线框内部分为本论文所设计的资源受限嵌入式环境多模型 AI 推理框架：

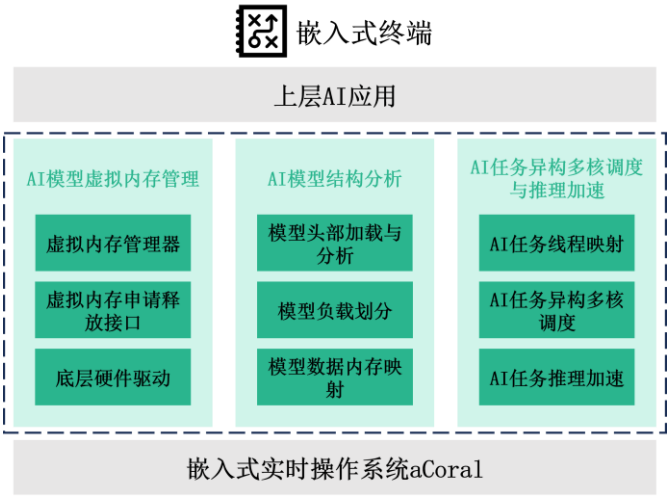


图 2-10 系统架构图

根据系统总体架构设计，AI 框架基于嵌入式实时操作系统 aCoral 运行，向上承接 AI 应用程序。框架由三个模块组成，分别是 AI 模型虚拟内存管理、AI 模型结构分析、AI 任务异构多核调度与推理加速，最终以一个目标检测追踪系统来验证上述模块的正确性：

（1）AI 模型虚拟内存管理负责所有 AI 模型的虚拟内存管理。该模块完成在无 MMU 的硬件环境下的虚拟内存管理机制，保证数据密集型的 AI 任务能够在有限的内存空间中完成推理。向上为 AI 任务提供虚拟内存申请释放的函数接口，向下依托于实时操作系统的内存管理，将 AI 任务内存管理与 RTOS 解耦，提高程序的健壮性与可移植性，并完成相关底层硬件的驱动初始化工作。

（2）AI 模型结构分析则是衔接前一个模块和后一个模块之间的桥梁，该模块完成两方面的工作。第一是完成模型的加载分析，从模型头部中解析出模型的结构信息。通过对这些信息的进一步解析，模块将初始化 AI 模型虚拟内存对象并完成模型的权重数据以及中间张量的虚拟内存分配。接着模块将根据 AI 模型虚拟内存

中国知网 https://www.cnki.net

对象初始化 AI 框架的模型运行时对象，完成 AI 任务初始化的工作；第二是根据 AI 模型的结构信息完成 AI 任务的负载划分，为后续 AI 任务在异构多核下的调度以及流水线加速方式提供 AI 任务的线程集合。

（3）AI 任务异构多核调度与推理加速模块是为了解决 AI 模型推理在多核情景下的资源分配问题。该模块的工作内容包括根据前一模块的 AI 模型负载划分结果进行 AI 任务线程映射、完成 AI 模型计算操作与逻辑控制操作在异构计算加速核心和 CPU 核心之间的调度策略以及 AI 任务在多核 CPU 下的流水线加速机制这三方面的工作。

2.5 本章小结

本章主要介绍了本文研究的资源受限嵌入式环境多模型 AI 推理框架所需的相关技术与设计，包括神经网络模型编译与优化方法，嵌入式虚拟内存管理技术、嵌入式 AI 应用开发平台以及框架的总体设计。

中国知网 https://www.cnki.net

第三章 AI 模型虚拟内存管理

本文所研究的资源受限嵌入式环境多模型 AI 推理框架的主要模块之一为 AI 模型虚拟内存管理。该模块是为了解决在无 MMU 的嵌入式环境中虚拟内存管理机制缺乏，导致 AI 应用难以部署的问题而提出的，主要方法是以软件的形式模拟硬件 MMU 的功能，针对 AI 模型完成一个虚拟内存管理器来管理 AI 模型数据。本章将从该模块的设计到具体的编程实现细节进行一一介绍。

3.1 AI 模型嵌入式环境部署分析

本文所研究的框架是为解决嵌入式环境中因资源受限无法完成 AI 模型推理的问题而设计的。通过研究目前主流的嵌入式 AI 推理框架，我们发现这些主流框架在嵌入式环境中进行 AI 模型推理前通常会有以下的处理过程：

（1）针对模型本身结构进行优化，比如使用剪枝、算子融合等方式，减小模型静态文件大小，方便在嵌入式端加载。

（2）针对模型数据类型进行优化，比如使用量化、蒸馏等方式，减小模型参数大小。

通过对这些框架进行 AI 模型推理时的分析，本文发现 AI 模型推理过程中的内存占用主要在于两个部分，分别是权重数据张量和中间计算张量。这两部分数据通常会在进行推理前申请到足够的内存池，后续推理中的张量数据操作都是在这些内存池中完成。

由以上分析可知，针对模型的前处理的最终目的都是为了减少模型在嵌入式端进行推理时的内存占用，尽可能避免因模型过大而无法在运行内存有限的嵌入式环境中无法完成推理。但是这些处理方式不仅牺牲了模型推理的精度，而且在一些特殊环境下依然无法解决嵌入式环境内存受限的问题。

所以本文设计的框架中借鉴虚拟内存管理机制，在无 MMU 的嵌入式环境中实现以有限物理内存加载 AI 模型并完成推理任务的目标。虚拟内存管理是一种二级存储管理机制，借助外部辅助存储空间，在有限的物理运行内存下为用户提供一个更大的连续虚拟地址空间（与辅助存储空间一样大小）。理论上如果这种机制融入到 AI 模型推理过程中，使用虚拟内存来管理模型权重数据张量和中间计算张量，那么就能在有限的物理内存中加载张量数据并完成模型推理。本文框架中的虚拟内存管理机制实现细节将在后续小节详细阐述。

中国知网 https://www.cnki.net

3.2 虚拟内存管理器设计

针对在无 MMU 嵌入式环境下，很多 AI 模型因为内存占用过大而无法在嵌入式设备部署并完成终端推理的问题[24]，本文提出一种使用软件方式模拟 MMU 功能的方法来解决嵌入式设备缺少虚拟内存机制的问题，从而使得超出硬件内存资源的神经网络模型也能在终端设备完成推理任务。这种以软件形式存在的虚拟内存管理器主要分为两个部分，分别负责模拟 MMU 的功能进行内存管理和底层存储硬件的驱动与通信支持。整体架构如图 3-1 所示。

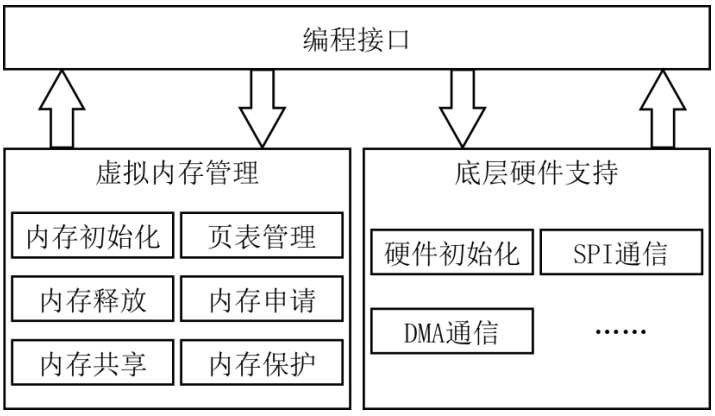


图 3-1 虚拟内存管理架构图

如架构图所设计的虚拟内存管理器，虚拟内存管理器的具体实现功能包含以下内容。

（1）底层硬件初始化。首先虚拟内存管理器需要对内部存储与外部辅助存储硬件完成初始化，包含存储硬件时钟初始化、数据通信硬件设置等内容。

（2）虚拟内存管理器初始化。管理器向操作系统申请一块连续内存，并对自身数据结构进行初始化，包含页面大小、物理页数、虚拟页表、页面访问顺序等内容的初始化设置。

（3）虚拟内存申请接口。虚拟内存管理器完成初始化操作后向用户提供内存申请接口，用户通过该接口申请到一定大小的虚拟内存空间的起始地址。

（4）虚拟内存访问接口。AI 模型进行推理时应使用该接口将虚拟地址转换为物理地址后进行后续操作。

（5）虚拟内存释放接口。用户通过此接口释放自己申请的虚拟内存。

（6）虚拟内存共享接口。用户可对申请到的虚拟地址进行共享设置，方便线程之间进行数据共享。

中国知网 https://www.cnki.net

（7）虚拟内存保护。未被设置为可共享的虚拟空间若被其他线程访问，管理器将返回错误或者空地址。

（8）页面置换算法。在用户访问虚拟内存时需要根据页表是否命中而采取相应的页面置换，页面置换算法可随用户需求灵活设置。

AI 模型虚拟内存管理器的正常使用流程如图 3-2 所示。

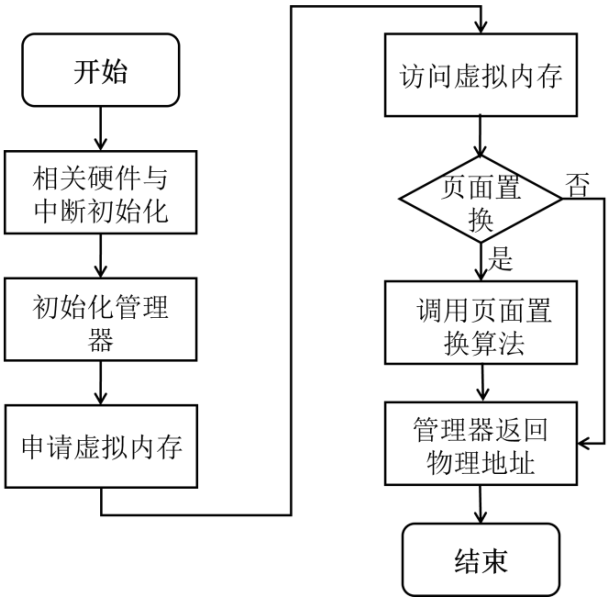


图 3-2 AI 模型虚拟内存管理器运行流程

在本文设计的框架中，用户申请内存成功后会得到一个虚拟地址，后续进行各种操作也是通过这个虚拟地址进行，所以虚拟地址的生成也是虚拟内存管理器的重要组成部分。这一部分定义了虚拟地址生成时各个控制字段的意义以及初始化，这关系到用户访问虚拟内存空间时的隐私性、安全性和共享性[25]，同时应该记录一些内存申请线程的信息方便进行管理。本文设置的虚拟地址结构如图 3-3 所示。

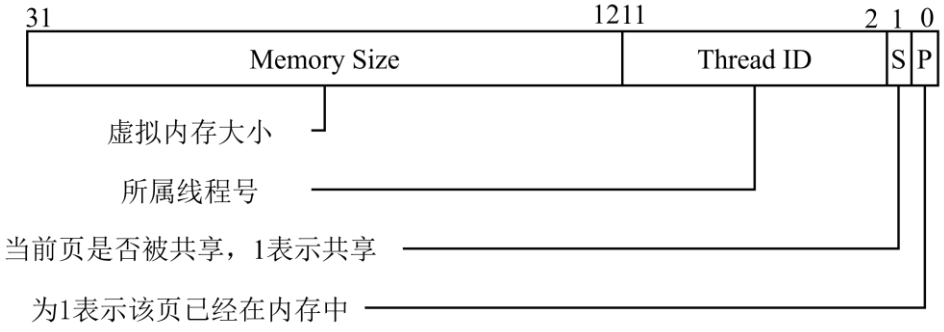


图 3-3 虚页页码控制字段

中国知网 https://www.cnki.net

虚拟地址长度为 32 位，第 0 位表示该虚页是否已经在物理内存中，为 1 表示虚页已经装入内存；第 1 位表示当前虚拟地址是否可以被其他地址共享，为 1 表示可以被共享；第 2-11 位表示该虚拟地址所属线程号；第 12-31 位则表示该虚拟地址所管理的内存大小；

当虚拟内存初始化完成以后，用户调用虚拟内存申请接口申请一定大小的内存空间时，虚拟内存管理器便会自动为用户初始化虚拟地址中图 3-3 所规定的控制字段内容。首先，虚拟内存管理器根据用户调用虚拟内存申请接口时传入的内存大小初始化虚拟地址的第 17-31 位。然后虚拟内存管理器根据操作系统提供的相关接口查询到当前进行虚拟内存申请的线程的线程号并以此初始化虚拟地址的第 7-16 位，记录线程号是为了方便虚拟内存管理器管理线程对虚拟地址的访问操作。最后第 1 和第 0 位分别被初始化为 0，只有当用户调用相关接口后才会被改变。

3.3 编程接口实现

3.3.1 虚拟内存初始化接口

虚拟内存管理器以软件的形式来模拟 MMU 硬件的功能，所以在使用虚拟内存管理的相关功能时需要先对虚拟内存管理器进行相关的初始化。本文所研究的框架中将 MMU 硬件功能抽象为一个单例类，硬件 MMU 所具备的属性和功能在这个类中以数据成员和成员函数存在，包括物理内存空间地址、物理内存空间大小、页面大小、页表等数据成员和虚拟内存管理器初始化、虚拟内存申请、虚拟内存释放、虚拟内存共享、虚拟内存保护、页面置换等成员函数，该类的定义如代码 3-1 所示。

类 virtual_mem 即为虚拟内存管理器的定义。其中 manager 指针为类 virtual_mem 的静态数据成员，是供用户使用虚拟内存管理机制各项功能的入口；physical_memory 指针指向虚拟内存管理器所使用的物理内存空间起始地址，该地址由实时操作系统分配得到；physical_memorysize 表示实际物理内存空间大小，由用户传入参数决定；page_size 表示页面大小，由用户传入参数决定，该成员与 physical_memorysize 决定了物理内存页面数量；used_pages 表示已使用的物理页面数量，用于在进行页面置换时判断当前物理页面是否已经全部被使用；page_table 表示页表，用于记录虚页号与实页号之间的映射关系；page_order 用于表示物理页面的访问顺序，用于页面置换时选择页面进行换进换出；此外 virtual_mem()等成员函数用于实现虚拟内存管理器的若干功能，比如管理器初始化、虚拟内存申请/释放接口、页码置换算法等。

中国知网 https://www.cnki.net

代码 3-1 虚拟内存管理器类定义

```
class virtual_mem {
private:
    static virtual_mem* manager;
    uint8_t* physical_memory;
    int physical_memorysize;
    int page_size;
    int used_pages;
    std::map<int, PageTableEntry> page_table;
    std::vector<int> page_order;
    virtual_mem(int memorysize, int pagesize);
public:
    uint8_t* allocate_virtual_mem(int size);
    uint8_t* access_virtual_mem(uint8_t* virtual_address,int flash_addr);
    uint8_t* free_virtual_mem(uint8_t* virtual_address);
    void share_virtual_mem(uint8_t* virtual_address_a, uint8_t* virtual_address_b);
    int page_replacement(int flash_addr);
    static virtual_mem* get_instance(int memorysize, int pagesize);
};
```

用户在使用虚拟内存管理的相关功能之前需要调用 get_instance()函数并传入相关参数进行初始化。用户调用 get_instance()函数的流程如图 3-4 所示。

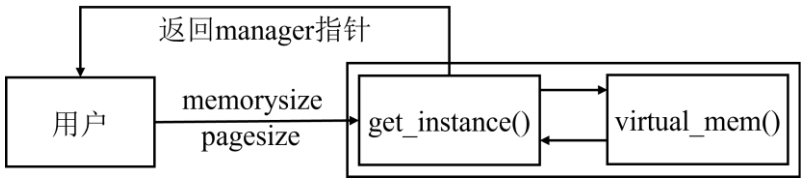


图 3-4 get_instance()函数调用过程

用户传递自定义的物理内存大小 memorysize 和页面大小 pagesize 参数给 get_instance()函数，get_instance()函数将使用这两个参数调用单例类的构造函数 virtual_mem()，在构造函数中将根据传入的参数初始化虚拟内存管理器的 physical_memory、physical_memorysize、page_size 等相关数据成员，完成类的实例对象初始化，接着 get_instance()函数将 manager 指针指向该对象并返回 manager 指

中国知网 https://www.cnki.net

针给用户，至此虚拟内存管理器初始化完成，用户可根据 manager 指针调用虚拟内存的相关功能进行后续开发。

3.3.2 虚拟内存申请接口

虚拟内存管理器初始化完成后会向上层提供虚拟内存申请接口，用户可通过 manager 指针调用接口函数 allocate_virtual_mem()并传入相关参数获取到虚拟地址，虚拟内存申请接口的调用过程如图 3-5 所示。

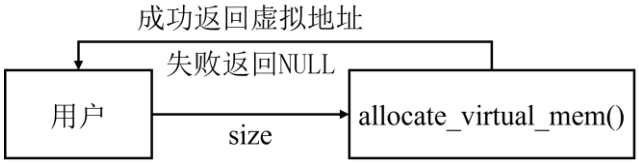


图 3-5 虚拟内存申请流程

用户调用虚拟内存申请函数 allocate_virtual_mem()后，函数根据用户传递的期望内存字节数进行内存分配，成功分配则返回虚拟地址，失败则返回 NULL 指针，函数的内部实现流程如图 3-6 所示。

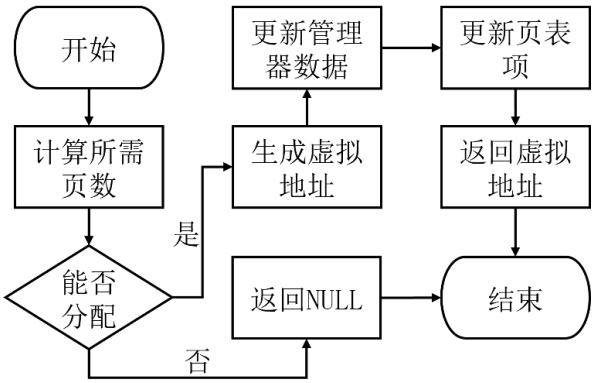


图 3-6 虚拟内存申请接口实现流程

（1）虚拟内存申请函数 allocate_virtual_mem()根据用户传递的内存字节数 size 以及虚拟内存管理器的 page_size 属性计算出用户申请内存所需要的物理页面数，若用户需要的内存页面数超过了管理器所管理的物理内存页面则返回 NULL，表示虚拟内存分配失败。

（2）若用户需求的页面数目能够满足，虚拟内存申请函数将初始化管理器设置的页码控制字段并生成正确的虚拟地址。

中国知网 https://www.cnki.net

（3）生成虚拟地址后，函数 allocate_virtual_mem()将对虚拟内存管理器的待申请虚拟地址、可用物理页面数目等相关属性进行调整。

（4）生成虚拟地址后，管理器的页表项将被函数 allocate_virtual_mem()更新，新的虚拟地址会被添加到页表中，并且对应的实页号会被置为 0，只有当用户对该虚拟地址进行读写访问的时候才会将辅助存储空间中的数据读入到物理内存中。

3.3.3 虚拟内存访问接口

用户通过虚拟内存申请接口申请虚拟内存成功后会拿到一个虚拟内存地址，用户需要对这块内存进行操作时需要通过 manager 指针调用虚拟内存访问接口函数 access_virtual_mem()来完成，虚拟内存访问接口的调用过程如图 3-7 所示。

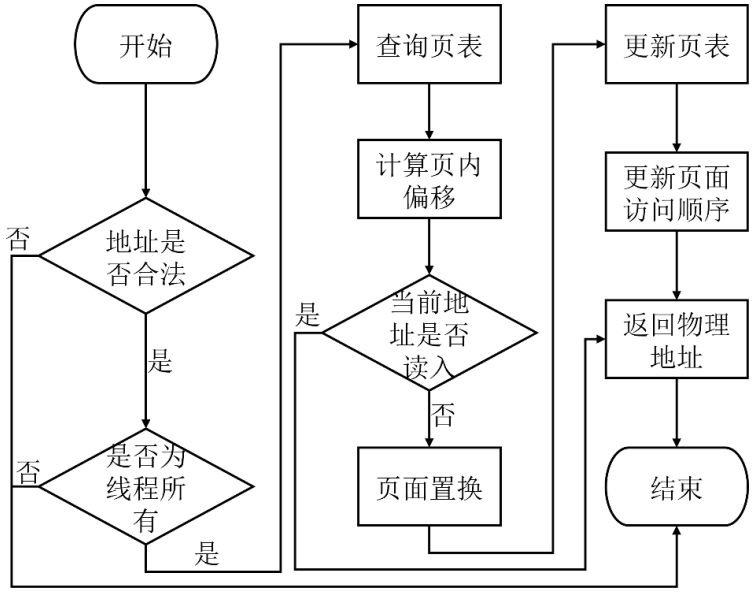


图 3-7 虚拟内存访问流程

（1）用户访问虚拟内存时，将自己的虚拟地址传递给虚拟内存访问接口函数 access_virtual_mem()后，接口首先检查虚拟地址的合法性，判断虚拟地址是否为虚拟内存管理器分配并且未回收的虚拟地址，如果地址非法则直接返回 NULL，不进行后续的操作。

（2）判断当前地址合法后，access_virtual_mem()函数会接着判断访问该地址的当前线程是否是该地址的拥有者，若不是则会返回 NULL。

（3）以上判断都合法后，该接口会根据地址中包含的信息计算该地址所映射的页面数量以及虚拟页号，并以此去查询页表中的内容，接着还会计算出页内偏移用于后续使用。

中国知网 https://www.cnki.net

（4）根据页表中页表项记录的虚页号与实页号，access_virtual_mem()函数会判断是否需要执行页面置换，若虚拟地址控制字段首位为 0 则表示线程想要访问的数据内容还未读入实际的物理内存当中，需要执行页面置换算法，如果不需要执行页面置换则根据页表中查询到的实页号以及页内偏移直接计算出实际物理地址并返回。

（5）若执行了页面置换算法，则 access_virtual_mem()会更新页表中的页表项，并且更新页面访问顺序，最后返回计算出的实际物理地址。

由以上虚拟内存访问接口的调用流程可知，在用户访问虚拟内存时，页面置换是十分关键的一环，它很大程度上影响了用户访问虚拟内存的效率。常见的页面置换算法有 FIFO（先进先出算法）、LRU（最近最久未使用算法）、OPT（最优置换算法）等[26]，由于本框架设计的虚拟内存管理服务是专门为 AI 模型推理进行设计的，所以考虑到 AI 推理的特殊性，框架使用 OPT 算法作为页面置换算法，上面三种算法的性能比较如图 3-8 和图 3-9 所示。

	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
FIFO	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页
LRU	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页	缺页
OPT	缺页	缺页	缺页	缺页	命中	命中	缺页	命中	命中	缺页	命中	命中	缺页	命中	命中	缺页

图 3-8 FIFO、LRU、OPT 算法页面访问情况

	FIFO	LRU	OPT
访问次数	16	16	16
缺页次数	16	16	8
缺页率	100%	100%	50%

图 3-9 FIFO、LRU、OPT 缺页率比较

为比较在 AI 模型在虚拟内存管理下不同页面置换算法的性能，本文设计了数据页面数为 4，物理页面数为 3，页面按序访问共 16 次的场景来比较 FIFO、LRU 和 OPT 算法的性能，结果如图 3-8 和图 3-9 所示。在本文设置的 AI 模型权重数据按序递增访问的情景下，FIFO 和 LRU 算法的缺页率达到了 100%，而 OPT 算法的缺页率则为 50%。尽管 OPT 算法在通用场景下难以实现，因为页面访问的顺序往往是难以预知的，但是在 AI 模型的推理过程中，数据的访问顺序是固定的，所以本文设计的虚拟内存管理器采取 OPT 算法作为页面置换算法，算法的实现细节如图 3-10 所示。

中国知网 https://www.cnki.net

（1）需要进行页面置换时，算法首先判断当前是否有空闲物理页，如果有空闲物理页分配则直接读取数据并返回物理页号，否则需要进行额外操作。

（2）如果物理页都已经分配完毕，算法就会根据未来的页面访问顺序取出最晚访问的虚拟页，并且将物理页中的数据保存到辅助存储中。

（3）得到新的空闲页号后，算法会将辅助存储中的数据读取到物理页中并且将物理页号返回。

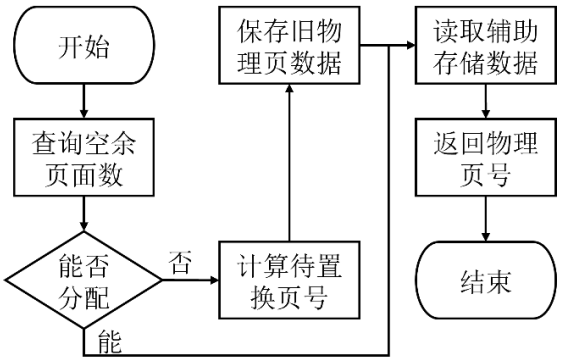


图 3-10 OPT 算法实现流程

3.3.4 虚拟内存释放接口

虚拟内存使用完毕后，用户应该及时进行内存释放，防止系统负担过重等问题。本文设计的虚拟内存管理器中为用户提供了虚拟内存释放接口，用户可以通过 manager 指针调用相应的函数进行内存的释放。该接口的调用过程如所示。

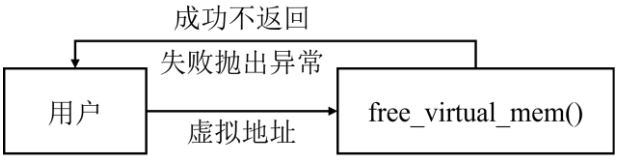


图 3-11 虚拟内存释放接口调用过程

用户通过 manager 指针调用虚拟内存释放接口函数 free_virtual_mem()时，需将待释放的虚拟地址作为参数传递给函数，接口函数成功释放地址后不会返回，若释放失败则会直接向操作系统抛出异常等待处理。该接口函数的内部实现流程如图 3-12 所示。

（1）线程通过 manager 指针调用 free_virtual_mem()并传入虚拟地址进行内存释放后，函数接口首先判断传入的地址是否合法，是否是由虚拟内存管理器分配的虚拟地址，若不是则向操作系统抛出异常不进行后续判断。

中国知网 https://www.cnki.net

（2）free_virtual_mem()函数继续判断当前释放地址是否为调用函数的线程所有，若不是则向操作系统抛出异常不进行后续判断。

（3）free_virtual_mem()函数接着判断该虚拟地址指向的物理地址是否启用了内存共享，若启用共享，则需判断当前释放操作是否为该虚拟地址所指向物理地址的最后一次释放，即是否还有其他线程在使用该虚拟地址所指向物理地址中的数据，若还有其他线程共享当前地址，则直接结束释放。

（4）若没有其他线程共享当前虚拟地址或者当前释放为该地址的最后一次释放，则更新页表并将虚拟地址所指向物理地址中的数据写回辅助存储中。

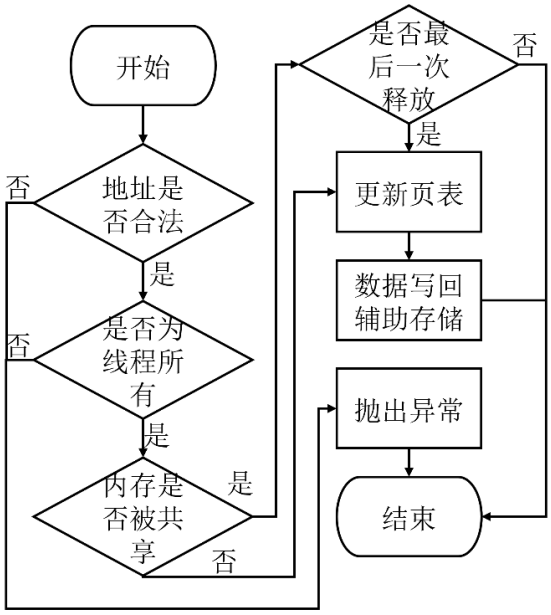


图 3-12 虚拟内存释放接口实现流程

3.3.5 虚拟内存共享接口

本文框架的虚拟内存管理机制还为用户提供虚拟内存共享功能，方便用户开发时进行线程间的数据共享。用户通过虚拟内存申请接口申请到虚拟地址后，可通过 manager 指针调用虚拟内存共享接口 share_virtual_mem()函数进行虚拟内存共享。该接口的调用过程如图 3-13 所示。

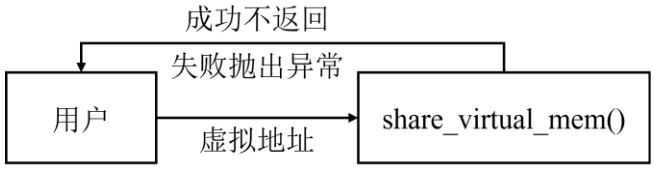


图 3-13 虚拟内存共享接口调用过程

中国知网 https://www.cnki.net

用户通过 manager 指针调用虚拟内存共享接口函数 share_virtual_mem()后，把需要启用共享机制的虚拟地址或者期望共享的虚拟地址作为参数传递给函数，函数根据用户传递的参数情况进行不同的处理，操作成功后函数不返回，失败则向操作系统抛出异常。该接口函数的内部实现流程如图 3-14 所示。

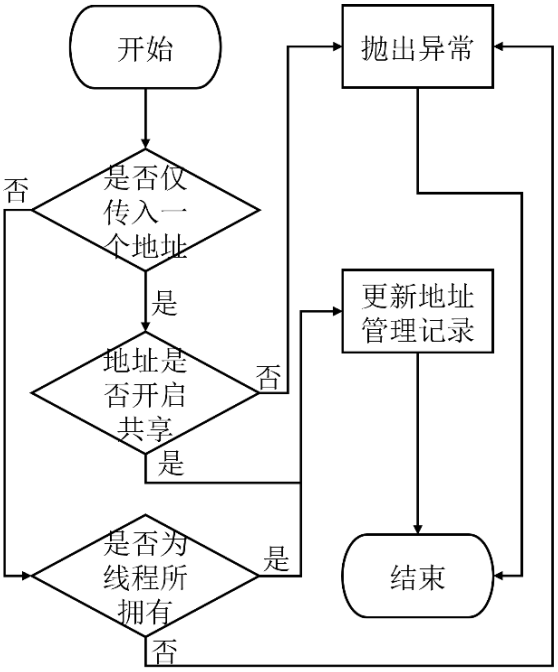


图 3-14 虚拟内存共享接口实现流程

（1）用户通过 manager 指针调用虚拟内存共享接口函数 share_virtual_mem()后可根据传入不同的参数量表明意图的操作。若只传入一个虚拟地址，则表示想要将该地址开启共享机制，此时如果接口判断调用线程为虚拟地址拥有者则可以更改虚拟内存管理器中记录的虚拟地址条目，更新地址管理记录，反之向操作系统抛出异常。

（2）用户如果传递两个参数，函数接口则判断为用户试图将第二个虚拟地址绑定到第一个虚拟地址，实现内存共享，此时如果第一个虚拟地址已经开启了共享功能则可以更改页表中的相关条目，反之向操作系统抛出异常。

3.4 底层硬件驱动实现

在虚拟内存管理机制中，除了 MMU 负责虚拟地址到物理地址的转换过程与管理工作，辅助存储硬件的驱动支持也是必不可少的一环，并且在很大程度上影响了虚拟内存应用的性能。本文实现的框架在硬件平台方面选择了 Kendryte 勘智公

中国知网 https://www.cnki.net

司所研发的的 K210 系统级芯片（SOC），这款芯片的辅助存储器选择的是 W25QXX 系列闪存芯片，这个系列的存储芯片是一种高性能的 SPI 接口 NOR Flash 存储器，被广泛应用于嵌入式系统、微控制器和工业自动化领域[27]。

在 K210 中使用的 W25QXX 闪存芯片是一块使用 SPI 进行连接控制的 NOR Flash 存储器，它在 K210 中的硬件连接如图 3-15 所示，WQ25XX 通过 6 个引脚与 SPI 连接，其中 DI、DO 为数据接收发送引脚分别连接到 SPI 的 MOSI 与 MISO 引脚，两根数据线保证了双方能以全双工的方式通信；CS 为片选引脚连接到 SPI 的 CS 引脚，通信时 SPI 通过该引脚选中设备；SCK 为时钟引脚，连接到 SPI 的 SCK 引脚用于同步通信时的时钟频率；HOLD 和 WP 引脚为控制引脚，连接到 SOC 中的 GPIO 引脚，通过拉低拉高 GPIO 电平来控制 WQ25XX 的一些状态。

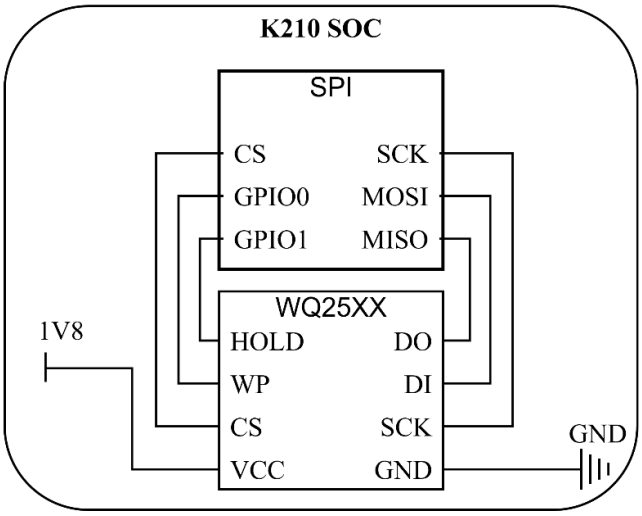


图 3-15 WQ25XX 硬件连接图

使用 SPI 与 WQ25XX 进行数据通信的流程如图 3-16 所示。首先需要选择 WQ25XX 进行连接的 SPI 端口，在 K210 中与 WQ25XX 硬件相连的是 SPI3 号总线，然后需要完成 SPI 的相关硬件配置，包括：工作模式的选择、帧格式配置、时钟频率配置和数据位宽配置。然后完成 W25QXX 的相关初始化，包括工作模式的选择，W25QXX 的工作模式包括 Mode0 和 Mode3，其中 Mode0 是常用的与 SPI 兼容的模式，一般初始化时都配置为 Mode0，只有在通信异常时尝试转换为 Mode3；接着需要绑定 W25QXX 的读写函数，本文实现了 W25QXX 的 SPI 模式页读写函数以及标准读写函数；完成这些配置后需要对 W25QXX 的芯片状态进行验证，只有验证成功后才能保证数据的读取与通信无误。所有的配置操作完成后用户可选择不同的通信模式，在完成所有的数据传输后，W25QXX 芯片向 CPU 发起中断，通知 CPU 完成后续工作[28]。

中国知网 https://www.cnki.net

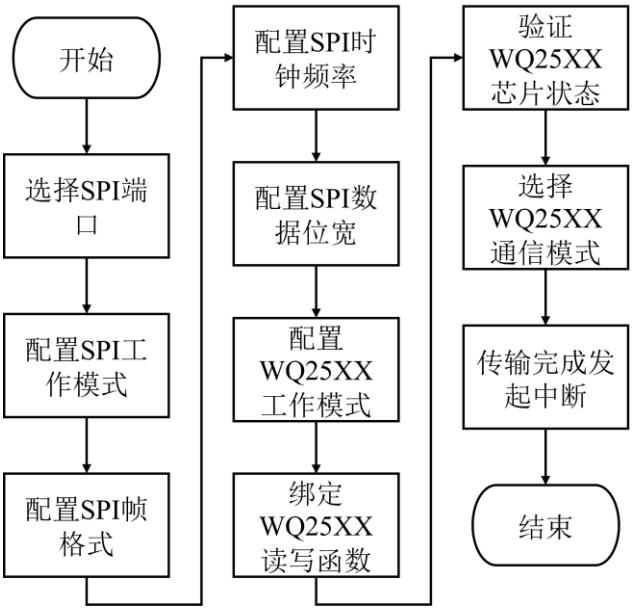


图 3-16 WQ25XX 通信流程

上述硬件配置以及初始化工作在框架中以如下驱动函数实现，用户可调用相关函数进行自定义配置以及编程时使用：

（1）typedef enum _w25qxx_status w25qxx_status_t 定义了芯片操作的返回枚举类型，包括正常、繁忙以及异常三种状态，用户需要根据这三种返回状态来判断操作后的结果；

（2）typedef enum _w25qxx_read w25qxx_read_t 是 W25QXX 芯片工作模式的枚举定义，包含 6 种工作模式：标准 SPI 读取、标准 SPI 快速读取、双线 DUAL 读取、双线 DUAL 快速读取、四线 QUAD 读取和四线 QUAD 快速读取，这些模式各有优劣，用户可根据实际应用场景选择合适的工作模式；

（3）w25qxx_status_t w25qxx_init(uint8_t spi_index, uint8_t spi_ss)：此函数完成 SPI 初始化以及 WQ25XX 读写函数的绑定，参数 spi_index 用于指明选择的 SPI 端口，参数 spi_ss 用于指定连接的片选引脚编号；

（4）w25qxx_status_t w25qxx_is_busy(void)：此函数用于判断 W25QXX 芯片当前状态是否繁忙，只有当 W25QXX 芯片处于空闲状态时用户才能发起通信进行数据传输操作；

（5）w25qxx_status_t w25qxx_chip_erase(void)：此函数用于擦除芯片中的所有数据，此外还提供有块擦除、页擦除等函数；

中国知网 https://www.cnki.net

（6）w25qxx_status_t w25qxx_enable_quad_mode(void) 和 w25qxx_status_t w25qxx_disable_quad_mode(void)：这两个函数用于启动与禁用四通道读取模式，四通道模式下数据的读取和写入速度更高，但是对系统的负担更大；

（7）w25qxx_status_t w25qxx_read_status_reg1(uint8_t *reg_data)、w25qxx_status_t w25qxx_read_status_reg2(uint8_t *reg_data) 和 w25qxx_status_t w25qxx_write_status_reg(uint8_t reg1_data, uint8_t reg2_data)：这三个函数用于读取和改写 W25QXX 芯片寄存器中的值，向用户提供芯片状态以及芯片控制功能，参数 reg_data 指向用于保存读取的寄存器值的内存空间，参数 reg1_data 和 reg2_data 指向用户向寄存器写入的值；

（8）w25qxx_status_t w25qxx_read_id(uint8_t *manuf_id, uint8_t *device_id)：读取芯片制造商 ID 和设备 ID，分别存储在参数 manuf_id 和 device_id 指向的地址中；

（9）w25qxx_status_t w25qxx_read_data(uint32_t addr, uint8_t *data_buf, uint32_t length, w25qxx_read_t mode)：此函数用于在读取指定地址中指定长度的数据，参数 addr 表示读取的地址，参数 data_buf 指向数据存储的内存空间，length 表示读取的数据字节数，参数 mode 表示启用的读取模式（标准、双通道和四通道）；

（10）w25qxx_status_t w25qxx_write_data(uint32_t addr, uint8_t *data_buf, uint32_t length)：此函数用于向 W25QXX 芯片指定地址中写入指定长度的数据，参数 addr 表示写入的地址，参数 data_buf 指向待写入数据的内存地址，参数 length 表示待写入数据的长度。

3.5 本章小结

本章主要介绍了本文研究的资源受限嵌入式环境多模型 AI 推理运行框架中虚拟内存管理机制的相关理论及设计，包括虚拟内存管理器设计、各种函数接口的设计与实现和底层硬件的驱动支持。

中国知网 https://www.cnki.net

第四章 AI 模型结构分析与负载划分

本文所实现框架中提出使用虚拟内存管理的方式管理 AI 模型，解决嵌入式环境中资源受限无法完成 AI 任务推理的问题，前一章中已经阐述了如何在无 MMU 环境中实现一套虚拟内存管理器，本章将接着介绍 AI 模型文件中的结构信息，并且对这些结构进行分析拆解用以完成 AI 模型的虚拟内存管理以及负载划分。

4.1 AI 模型加载与分析

在以往的嵌入式 AI 应用开发中，神经网络模型通常是存储于外部存储器中，比如 NOR Flash 中，每当 AI 应用启动的时候需要将整个模型文件读入运行内存中，并完成 AI 模型运行时对象初始化，即对 AI 模型的各种结构信息进行加载与分析。为了避免 AI 神经网络模型文件过大无法直接全部载入运行内存中，在本文所设计的框架中使用虚拟内存管理机制来管理整个神经网络模型，为了做到这一点就需要为虚拟内存管理设计一套 AI 模型加载与分析的适配方案。

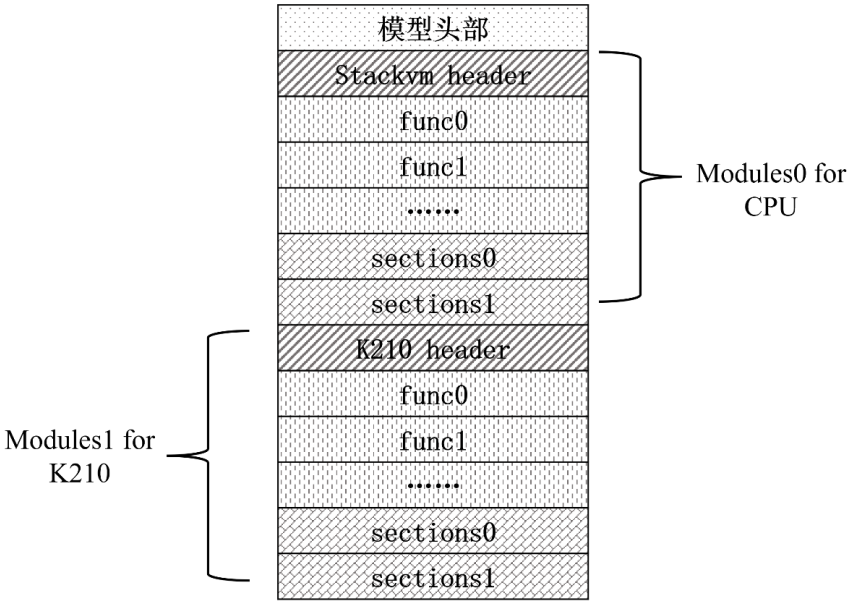


图 4-1 KMODEL 文件结构

本文框架所使用的 AI 模型是由 nncase 神经网络编译器编译转换而来的文件，进行模型加载与分析即是对模型文件的结构信息进行解析。经过 nncase 编译后的神经网络模型 Kmodel 文件的数据结构如图 4-1 所示。

AI 模型的 Kmodel 文件存放于 WQ25XX 闪存芯片中，当 SOC 上电以后启动 AI 应用时才会加载进内存进行 AI 推理。文件结构主要分为三大部分，分别是模型

中国知网 https://www.cnki.net

头部（model_header）、CPU 运算模块（Stackvm_module）和 KPU 运算模块（K210_module）。

在本文框架中针对 AI 模型管理设计了虚拟模型类（virtual_model）来管理记录 AI 模型的各种结构信息，其定义如代码 4-1 所示。

代码 4-1 virtual_model 类定义代码

```
class virtual_model
{
private:
    int model_flash_addr;
    std::vector<virtual_module_struct> module_;
    uint8_t *model_data_;
    int virtual_flag = 1;
    void modules_info();
    void text_process(std::set<int>& rdata_op_offset,std::set<int>&
data_op_offset);
    void offset_to_blocks(std::set<int>& op_offset,std::vector<virtual_block>&
blocks);
    int weights_flash_address(const lea_buffer_op_t &op);
public:
    virtual_model(int model_size,int flash_addr);
    const std::vector<virtual_module_struct>& get_modules();
    int get_flag();
    uint8_t *get_model_data();
    ~virtual_model();
};
```

AI 模型的虚拟模型管理类如类 virtual_model 所示。其中 model_flash_addr 记录了模型的 Kmodel 文件在外部存储器中的存放地址，用于模型加载时进行数据读取；module 成员使用 vector 容器记录了模型的各个模块内容，方便用户管理各个模块；model_data 用于记录模型文件载入内存时的地址，只有当 virtual_flag 成员为 0，即不启用虚拟模型管理 AI 模型的情况下，model_data 才会生效；modules_info()函数用于分析出模型文件的各个模块信息并存入 module 容器中；text_process()、offset_to_blocks()和 weights_flash_address()三个函数共同完成 AI 模型的各个模块

中国知网 https://www.cnki.net

的组成部分的分析与拆解；get_modules()和 get_model_data()函数则是 virtual_model 类对外暴露的接口，供用户获取 AI 模型的各个模块以及完整二进制数据。

用户调用 virtual_model 类构造函数实例化 AI 模型虚拟管理对象时，需要将模型的字节数以及在外部存储器中的存储地址作为参数传入构造函数，构造函数会在内部完成 AI 模型虚拟管理对象的全部初始化，构造函数的调用过程如图 4-2 所示。

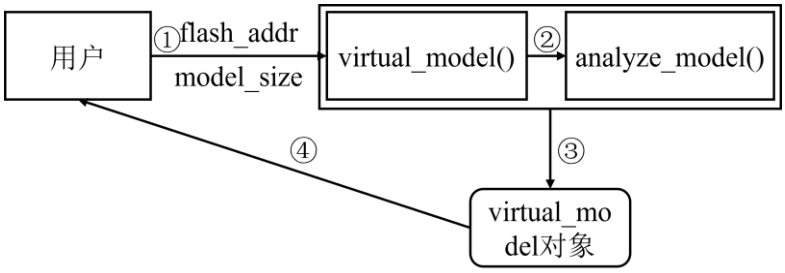


图 4-2 virtual_model 调用过程

用户将模型存储地址 flash_addr 和模型大小 model_size 传递给 virtual_model 构造函数后，构造函数会读取模型文件头部数据，验证无误后会调用模型分析函数 analyze_model()开始 AI 模型的拆解分析过程，分析完毕后会将初始化完成的 virtual_model 对象返回给用户，用户可以使用该对象进行后续的 AI 模型推理过程。

4.1.1 AI 模型头部结构

AI 模型头部位于 Kmodel 文件的起始位置，共占 32 字节，保存有模型的一些结构信息，包括模型的版本号、字节对齐信息、模块数、入口模块和入口函数等内容，其定义如代码 4-2 所示。

结构体 model_header 即为模型的头部定义由 8 个 uint32_t 数据成员组成。其中 identifier 成员用于验证模型类型是否为 Kmodel 文件，便于后续调用相应的模型加载函数；version 成员表示模型的版本号，需要与 AI 运行时环境版本号相对应，目前框架适配的版本号为 1.9.0.0 版本；header_size 成员表示头部的长度，当前版本模型头部长度为 32 字节；flags 成员用于标识当前模型文件运行环境，为 0 表示在嵌入式端，为 1 表示在 PC 端；alignment 成员表示模型文件的对齐方式，当前版本下模型文件对齐方式为 32 字节对齐；modules 表示本模型文件共有几个模块，一般情况下每个模型文件都有两个模块，分别是 CPU 运算模块（Stackvm_module）和 KPU 运算模块（K210_module）；entry_module 用于标识入口模块，模型加载完

中国知网 https://www.cnki.net

毕开始推理时会从入口模块开始；entry_function 用于标识入口函数，入口模块中的入口函数才是模型推理的真正入口。

代码 4-2 模型头部定义代码

```
struct model_header
{
    uint32_t identifier;
    uint32_t version;
    uint32_t header_size;
    uint32_t flags;
    uint32_t alignment;
    uint32_t modules;
    uint32_t entry_module;
    uint32_t entry_function;
};
```

4.1.2 AI 模型运算模块结构

在 AI 模型的 Kmodel 文件中，紧接模型头部后的就是第一个模块的内容，第一个模块通常为在 CPU 上进行推理计算的模块（即 stackvm_module）。各个模块尽管长度不一样，但都有相同的组成部分，模块的组成如图 4-1 所示，每个模块都由一个头部，若干个 functions 和 sections 组成。

每个模块所占字节数不固定，但是都有长为 48 字节的模块头部记录该模块的详细结构信息，模块头部的定义如代码 4-3 所示。

结构体 module_header 即为模块头部的定义，其由一个长为 16 字节的 module_type_t 成员以及 8 个 uint32_t 成员组成。其中 type 成员长为 16 字节，用于表示当前模块的名字，例如 stackvm、k210 等；version 成员表示当前模块的版本，不同版本的模块组成结构有所差异，在本文框架中的模块版本为 1.0.0 版本；header_size 记录头部的长度，即 48 字节；size 成员记录整个模块所占字节数，包含头部长度在内；mempools 记录该模块所使用的私有内存池数量，这些内存池通常用于记录中间计算结果或者临时变量；shared_mempools 记录该模块的共享内存池数量，共享内存池通常用于多个模块之间的数据交换；sections 用于记录模块中数据段的数量，这些数据段通常包含权重数据段和算子数据段；functions 记录了模

中国知网 https://www.cnki.net

块中计算函数的数量，每个计算函数通常完成一个或多个神经网络运算；reserved0是保留字节，以备未来模块可能的扩展。

代码 4-3 模块头部定义代码

```
struct module_header
{
    module_type_t type;
    uint32_t version;
    uint32_t header_size;
    uint32_t size;
    uint32_t mempools;
    uint32_t shared_mempools;
    uint32_t sections;
    uint32_t functions;
    uint32_t reserved0;
};
```

紧接着模块头部的内容是该模块内存池的描述字段，这些字段记录了模块内存池的类型以及内存池大小，其定义如代码 4-4 所示，共占 8 字节，其中 location 用于表明内存池的类型，为 0 表示输入张量内存池，为 1 表示输出张量内存池，为 2 表示权重张量内存池，为 3 表示中间计算张量内存池，为 4 表示共享张量内存池；reserved0 是一个长为 3 的保留字段数组；size 用于表示该内存池所需要的字节数大小，用于后续内存空间的分配。

代码 4-4 内存池描述字段

```
struct mempool_desc
{
    memory_location_t location;
    uint8_t reserved0[3];
    uint32_t size;
};
```

在内存池描述字段之后是若干个计算函数描述块，数量由模块头部（module_header）中的成员变量 functions 决定，每个计算函数描述块大小不一，内

中国知网 https://www.cnki.net

容也不尽相同，但是都有共同的结构组织，计算函数描述块的结构组织如图 4-3 所示。

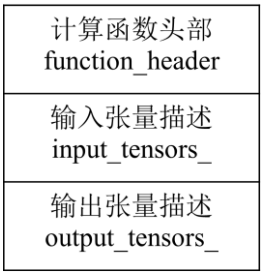


图 4-3 计算函数组织结构

计算函数描述块由三部分组成，分别是计算函数头部（function_header）、输入张量描述（input_tensors_）和输出张量描述（output_tensors_）。在计算函数头部中包含了各种函数计算时需要的信息，如代码 4-5 所示，结构体 function_header 即为计算函数头部定义。计算函数头部大小（header_size）表示头部所占字节，计算函数块大小（size）表示整个计算函数描述块的大小，用于模型加载分析时读取单个计算函数的全部内容，输入张量内存池大小（input_pool_size）和输出张量内存池大小（output_pool_size）则是表示输入输出张量的大小，方便在进行 AI 推理前为输入输出数据分配足够的内存空间，复杂模型输入输出张量往往有多个，所以输入张量个数（inputs）和输出张量个数（outputs）则是用于记录这些数据的变量，在模型文件中，每个计算函数的操作算子都是统一存储，所以需要用计算函数入口地址偏移（entrypoint）记录函数的操作入口，并且使用计算函数算子数据段大小（text_size）记录计算函数的算子数据段大小；

每个计算函数描述块头部之后都紧跟着该计算函数输入输出张量信息，它们都用相同的数据结构进行描述，如代码 4-6 所示。结构体 inout_tensor_info 就是用于描述输入输出张量的数据结构。其中 shape 用于表示张量的维度信息，例如在图像处理的神经网络模型中输入张量通常是一个四维张量（例如[1，3，224，224]）；strides 指定了该张量在每个维度上的步长，记录了张量的内存布局方式；range 则是记录张量所在内存池的一些信息，比如内存池类型、数据类型（uint_8 或者 float_32）、张量共享模块、内存起始地址和张量大小等；bind_tensor 表示外部输入到模块中的张量；staging_tensor 表示暂存张量，张量可能在异构计算设备中拷贝；device_tensor 表示存储在实际推理计算设备上的张量（比如 CPU、NPU 等处理核心）；

中国知网 https://www.cnki.net

代码 4-5 计算函数头部代码

```
struct function_header
{
    uint32_t header_size;
    uint32_t size;
    uint32_t input_pool_size;
    uint32_t output_pool_size;
    uint32_t inputs;
    uint32_t outputs;
    uint32_t entrypoint;
    uint32_t text_size;
};
```

代码 4-6 输入输出张量描述代码

```
struct inout_tensor_info
{
    runtime_shape_t shape;
    runtime_shape_t strides;
    memory_range range;
    runtime_tensor bind_tensor;
    runtime_tensor staging_tensor;
    runtime_tensor device_tensor;
};
```

在若干个计算函数描述块之后，会有数个字节的 0 字节补齐，接着便是模块中的数据段，这部分数据是每个模块最后的内容，包含模块权重数据（.rdata）和算子结构信息（.text）两个部分。这两个部分都有相同的组成部分，如图 4-4 所示，在数据段的起始位置存放的是数据段头部，头部存储了该数据段的各种信息，其数据结构如代码 4-7 所示。

结构体 section_header 即为数据段头部的定义，其中 name 字符数组存放了该数据段的名字，当前版本下有.rdata 和.text 两种名字，分别表示权重数据段和算子数据段；flags 用于表示模块所属计算环境，例如（CPU、NPU 等）；body_start 表

中国知网 https://www.cnki.net

示数据段主体部分的起始地址偏移;body_size 表示数据段主体部分的字节数;reserved0 属于保留字段，用于后续版本扩展;

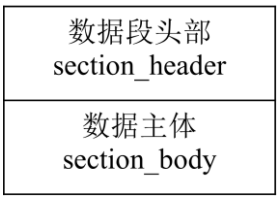


图 4-4 数据段组织结构

代码 4-7 数据段头部代码

```
struct section_header
{
    char name[MAX_SECTION_NAME_LENGTH];
    uint32_t flags;
    uint32_t body_start;
    uint32_t body_size;
    uint32_t reserved0;
};
```

在当前版本下，AI 模型经过 nncase 神经网络编译器编译优化后得到的 Kmodel 文件除了模型头部外就是若干模块，除了上面介绍的 CPU 运算模块（stackvm_module）外就是 KPU 运算模块（K210_module），这一模块所包含的推理过程都是在 K210 上的 KPU 中完成，其组织方式与数据结构定义都与 CPU 运算模块类似，只是各个数据成员值有所不同，此处不再详细叙述。

4.1.3 AI 模型加载分析

根据上两小节对 AI 模型 Kmodel 文件数据结构的描述，本文实现的模型加载分析针对这些内容做了若干数据结构实现以便模型加载。由于 Kmodel 文件中模型头部所占字节仅为 32 字节，所以本文设计中直接将这一部分按照 4.1.1 小节中所描述的数据结构记录在内存中，此处不再详细叙述。

对于模型文件中剩余模块由于所占字节数不确定并且通常体积较大，所以需要设计额外的数据结构对存储在外部辅助存储器中的模型文件进行加载分析，方便后续进行 AI 模型的虚拟内存管理。如代码 4-8 所示，结构体 virtual_module_struct 即为记录辅助存储器中 Kmodel 文件中各个模块结构信息的定义。其中 header 表

中国知网 https://www.cnki.net

示该模块的头部内容，记录该模块头部的字节大小、起始地址以及剩余字节内容；mempool 则记录了该模块的权重张量数据大小、中间计算张量大小以及它们在辅助存储中的起始地址偏移；func 记录了模块中所有计算函数的字节数之和以及第一个计算函数起始地址的偏移；text 则记录了模型文件中所有算子信息数据段的大小以及起始地址偏移；rdata 则是用于记录权重张量数据段的相关信息；rdata_blocks 和 data_blocks 容器是用于记录权重张量数据与中间计算张量的分块信息，这部分内容在后续进行 AI 模型虚拟内存映射时起重要作用，详细内容将在 4.2 和 4.3 小节中阐述。

代码 4-8 虚拟 AI 模型模块数据结构

```
struct virtual_module_struct
{
    virtual_module_header header;
    virtual_module_mempool mempool;
    virtual_module_func func;
    virtual_module_section_text text;
    virtual_module_section_rdata rdata;
    std::vector<virtual_block> rdata_blocks;
    std::vector<virtual_block> data_blocks;
};
```

进行 AI 模型加载分析时，首先完成的是模型头部的加载，需要从头部中获取模型的模块结构信息，然后才能完成后续的加载过程。根据上述模型头部结构，本文框架设计的 AI 模型头部加载流程如图 4-5 所示。

virtual_model 类构造虚拟 AI 模型对象时，首先根据用户传入的 flash_addr 地址参数从外部存储中读取 AI 模型文件起始 32 字节长度的内容，即模型头部。接着根据模型头部的数据验证 identifier 和 version 是否合法，在本文框架版本中这两个变量的值分别为“KMDL”和“1900”（版本号为 1.9.0.0）。

验证模型合法后，构造函数便将模型的一些结构信息记录到 virtual_model 类中的数据成员中，包括模型大小、模型文件对齐方式、模型模块数目、模型入口模块和模型入口函数等信息。

完成模型头部内容的加载分析后，根据 4.1.1 和 4.1.2 小节以及上述数据结构的实现，在进行 AI 模型虚拟内存管理时的模型加载分析流程如图 4-6 所示。首先需要从读取的模型头部中根据 modules 变量获取模型的模块数量，然后根据每个

中国知网 https://www.cnki.net

模块的头部信息中的 functions 变量得到计算函数的数量，根据这些计算函数头部中的 size 计算出全部计算函数的字节数，计算函数读取完毕后根据模块头部中的 sections 变量依次处理所有的数据段，根据数据段头部中的 name 变量和 body_size 分别处理权重数据段（.rdata）和算子结构数据段（.text），当所有的模块按照上述流程处理完毕后，整个 AI 模型 Kmodel 文件的结构信息就加载分析完毕，可以进行后续的算子与权重分析工作。

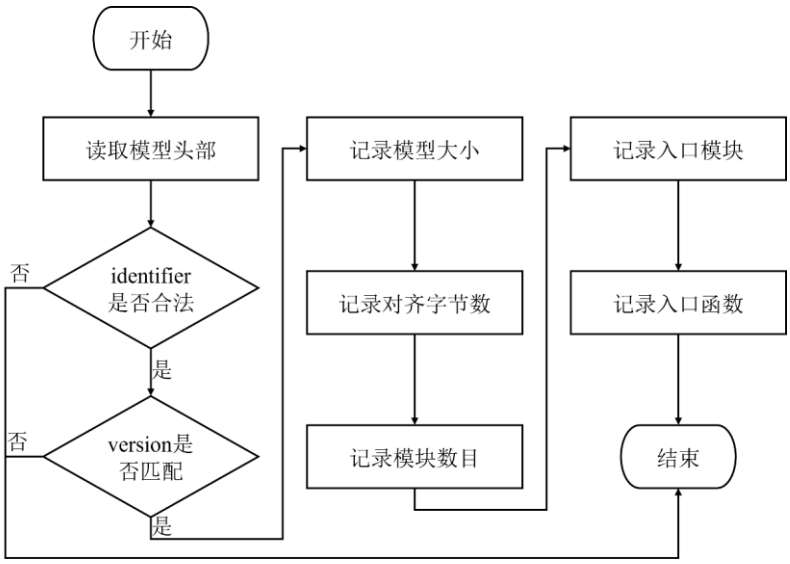


图 4-5 AI 模型头部加载流程

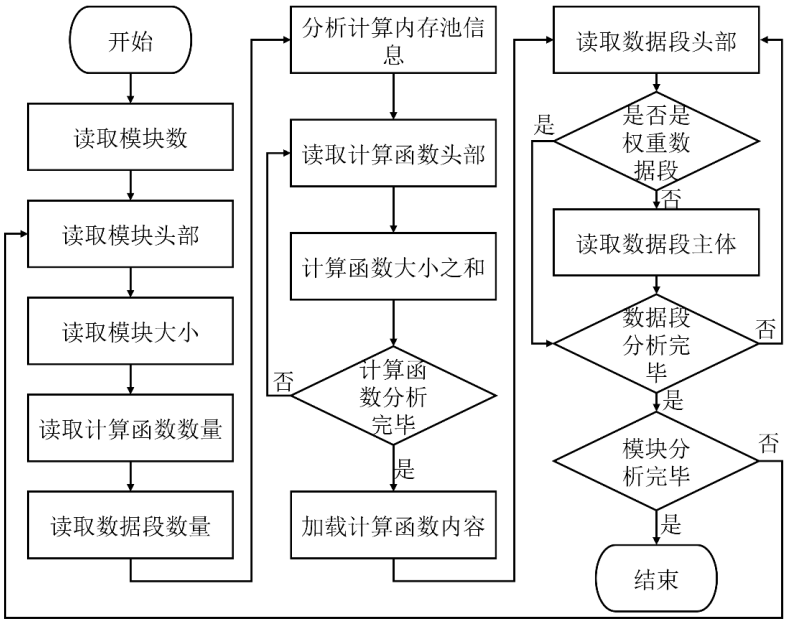


图 4-6 AI 模型加载与分析流程

中国知网 https://www.cnki.net

4.2 AI 模型算子与张量结构分析

完成 AI 模型的结构解析后，就可以通过上一小节得到的信息进行模型算子结构和权重数据的拆解分析。通过调研发现，在嵌入式端进行 AI 模型的推理计算中，占用物理内存最大的部分是权重数据张量以及中间计算张量，这部分数据占整个模型运行时内存大小约 90%的内容[29]，所以在本框架中的 AI 模型虚拟内存管理主要是针对这两部分数据进行处理。

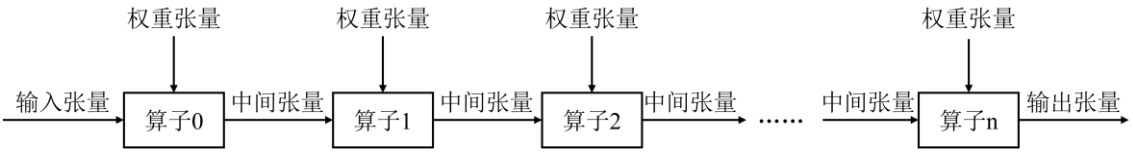


图 4-7 算子、张量之间的关系

不管是权重数据张量还是中间计算张量，它们都是和算子操作密切相关的，这三者之间的关系如图 4-7 所示。模型进行推理计算时，从第一个算子开始，接受输入张量和权重数据张量的输入，进行计算后得到中间计算张量输出，并将这个中间张量作为下一个算子的输入，剩余算子按序依次计算直到最后一个算子执行完毕得到模型的输出张量。

根据上述算子与张量数据之间的关系可知，如果要对权重数据张量和中间计算张量进行处理并完成虚拟内存管理，就需要从算子结构入手。由 4.1.2 小节中模型模块的组成部分可知，AI 模型在进行模型推理时，是从入口模块中的入口函数开始执行的，而入口计算函数所包含的算子操作又是以.text 数据段的方式存储描述的，所以模型的算子结构信息都保存在数据段.text 中。

在 nncase 神经网络编译器编译优化后的 AI 模型 Kmodel 文件中，各种算子信息存放在数据段.text 中，但数据段.text 中还包含其他内容，一个算子描述只指明了数据的处理计算方式，而这些数据的来源以及去处等控制操作都不包含在算子描述中。这些数据的读取与输出操作在数据段.text 中以各种控制指令描述，比如 LDNULL（加载空值 NULL）、LDC_I4（加载 4 字节常量）、LDIND_I4（从指针中加载 1 字节带符号整数）、LEA_BUFFER（加载指针地址）等等。这些控制指令完成了所有张量数据的来去控制，而这些张量在流动中需要一个中转站来存放数据以便各种算子读取自己需要的张量数据进行计算，在本文框架的版本下，这个中转站用一个栈结构来实现，所有数据存放都是入栈，所有数据的读取都是出栈，其流动过程如图 4-8 所示。

中国知网 https://www.cnki.net

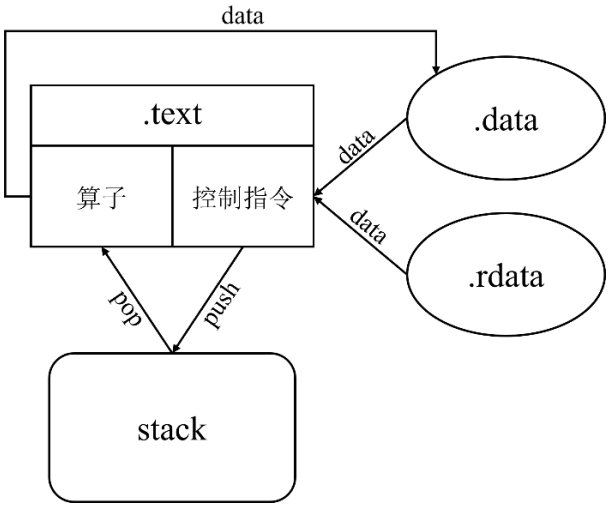


图 4-8 张量数据流动过程

AI 模型 Kmodel 文件进行推理时，权重张量数据以静态的方式存储于数据段.rdata 中，而中间计算张量会由于输入张量的不同而动态变化[30]，所以只是在初始化 AI 模型推理对象时申请一块内存池来预留这些中间计算张量的空间。所有的张量在数据段.text 中都记录有其在各自内存池中的起始地址，如果要分析每个算子所使用的张量信息，就需要从数据段.text 中解析出每一个与张量数据控制相关的指令。根据以上张量数据流动的分析，本文框架中对 AI 模型算子与张量分析的流程设计如图 4-9 所示。

（1）从入口函数描述块中 entrypoint 变量记录的数据段.text 入口开始读取操作码，如果读取到与张量数据存取相关的控制指令则进行后续操作，反之跳过当前控制指令继续读取下一个操作码；

（2）读取到与张量数据存取相关的控制指令后，根据指令中记录的压栈张量数据起始地址偏移得到张量数据在数据段.rdata 或.data 中存储的地址，将这个地址信息记录；

（3）记录一次控制指令携带的张量数据地址信息后，继续读取操作码，直到读取到计算算子的操作码为止；

（4）读取到一次计算算子的操作码时表明该算子所使用的张量数据地址信息已经全部分析完毕，需要完成一次算子的张量信息记录；

（5）如果已经处理到当前计算函数所拥有的数据段.text 内容末尾则结束当前分析，反之返回（1）继续上述操作。

中国知网 https://www.cnki.net

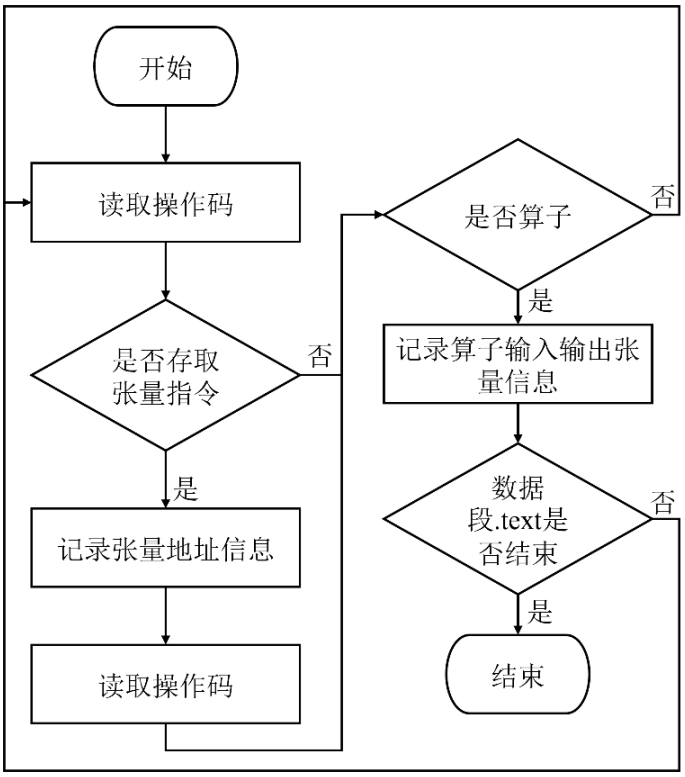


图 4-9 数据段.text 分析流程

4.3 AI 模型虚拟内存映射

本文框架提出的 AI 模型虚拟内存管理机制最核心的地方就在于使用虚拟内存管理的方式使得 AI 模型推理过程中内存占比最大的各种张量数据能够在有限的物理内存空间中完成推理计算。根据上小节分析得到的算子张量信息，框架就可以利用第三章中实现的虚拟内存管理器来完成这些张量的虚拟内存映射。在对这些张量内存池进行虚拟内存映射前还需要对其做一些处理。

为了保证在进行一次算子计算时所使用的张量能同时存在于物理内存中而不会因为页面置换导致某一些张量没有被读入内存，框架需要先对这些张量进行内存分块，然后为这些张量内存块分配虚拟内存，只要保证每个内存块的大小小于等于虚拟页面大小就能保证在访问一个内存块中的数据时不会产生页面置换，那么一个虚拟页面的大小限定也就需要由算子决定，一个页面的最小值是使用张量数据最多的算子所占用的内存大小。基于以上限制条件决定了张量内存块大小（block_size）、页面大小（page_size）和最大算子（这里指使用张量内存占用最大，max_op_size）之间的关系是：max_op_size≤block_size≤page_size，为了方便实现，

中国知网 https://www.cnki.net

本文实现的框架中将内存块大小和页面大小设置为一样，设置为 1MB 大小，若用户模型中的最大算子（max_op_size）大小大于这个值，可以自行修改页面大小。

根据以上分析，框架进行 AI 模型虚拟内存映射时的实现流程如图 4-10 所示。

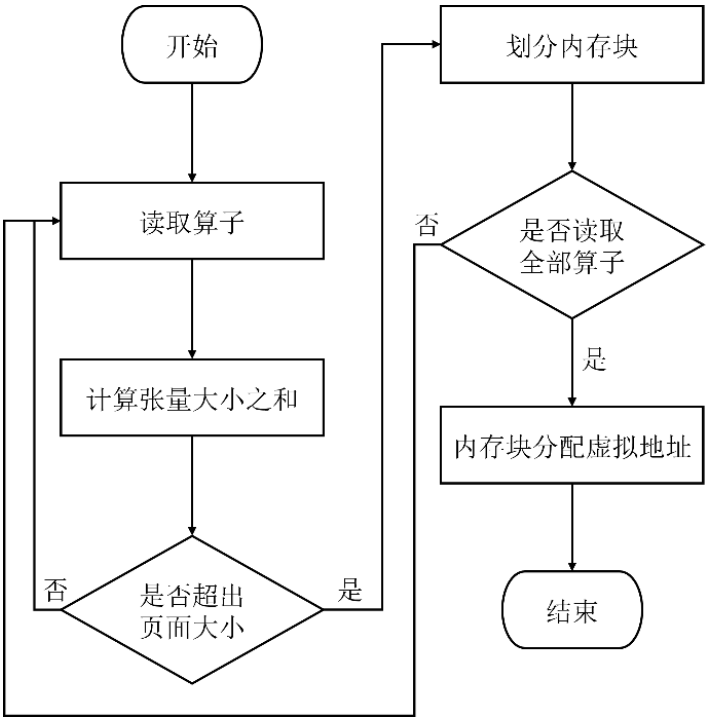


图 4-10 AI 模型虚拟内存映射过程

（1）根据 AI 模型算子与张量分析结果读取算子信息，计算出该算子的张量数据内存大小之和，并累加之前算子的张量数据内存大小之和，如果还未超出页面大小且不是最后一个算子则循环读取下一个算子信息；

（2）若累加结果超出页面大小，则将之前累加结果作为一个内存块切分，并继续读取下一个算子，重复（1）的工作；

（3）当读取完最后一个算子信息，切分完所有内存块后便使用虚拟内存管理器提供的接口为所有的内存块分配虚拟地址。

至此 AI 模型的结构分析以及虚拟内存管理工作就已经全部完成。

4.4 AI 任务负载划分

对 AI 模型的加载分析不仅仅是为了实现 AI 模型的虚拟内存管理，更是为了将 AI 模型的推理过程细分透明，使得 AI 任务在实际硬件平台中能更好的利用硬件资源，提高推理效率，所以本框架结合前面的工作内容对 AI 任务的推理过程作

中国知网 https://www.cnki.net

出负载划分，将一套 AI 模型推理流程分成不同的模块，并将这些模块映射到具体的硬件资源进行运算，并在 K210 硬件平台中实现具体实例。

在嵌入式端应用中，任务的负载划分通常是指在不同 CPU 核心中的负载划分[31]，而在 AI 任务中通常还涉及到异构加速计算器的使用以及异构核心之间大量数据的吞吐处理。框架实现的负载划分方案以 K210 硬件平台为示例，在进行 AI 模型推理时主要涉及两个 CPU 核心和一个 KPU 计算加速单元的工作。在一般的神经网络模型推理中可以大致将推理流程分为三部分：数据预处理、神经网络模型主干网络计算和输出数据后处理过程[33]。由此在 K210 平台中可以有一个大概的 AI 任务负载划分策略，整体的负载划分策略如图 4-11 所示。

AI任务负载划分		
CPU0	KPU	CPU1
任务调度、数据处理	主干网络的加速计算	输出后处理

图 4-11 K210 AI 任务负载划分

在具备 AI 计算硬件加速器的硬件平台中，AI 任务主要包含数据预处理与逻辑控制、加速硬件加速主干网络计算和输出后处理三部分工作。所以结合 K210 的硬件条件（两颗 CPU 和一颗 KPU）可以将 AI 任务按照以下内容进行负载划分：

（1）在 CPU0 中负责完成 AI 任务的任务拆分与调度以及输入数据预处理过程（比如数据格式转换、数据归一化、图像缩放等）；

（2)在神经网络处理单元 KPU 中负责主干网络的加速计算，比如卷积层计算、全连接层计算和激活函数计算等内容；

（3）在 CPU1 中完成模型输出的后处理工作，比如 Softmax 计算、非极大值抑制操作、解析检测框等。

AI 任务的线程拆分映射以及负载调度实现细节等内容将在第五章中详细阐述。

4.5 本章小结

本章主要介绍了本框架中 AI 模型 Kmodel 文件的组织结构以及框架针对实现 AI 模型虚拟内存管理而对模型进行的加载分析过程，最后提出了根据具体硬件平台和 AI 模型结构的 AI 任务负载划分策略。

中国知网 https://www.cnki.net

第五章 AI 任务异构多核调度与推理加速

在前面的章节中已经完成 AI 模型的虚拟内存管理以及负载划分工作，而这些内容如何融入到嵌入式实时操作系统（RTOS）中的问题还未解决。本章将从 AI 任务线程的映射过程、AI 任务在异构多核下的调度策略以及 AI 任务在多核下的流水线并行加速方法进行阐述，以上内容都基于 K210 硬件平台进行实现。

5.1 AI 模型推理任务线程映射

根据第四章中 AI 模型文件的结构分析可知，在 Kmodel 文件中，模型的算子结构都存放在数据段.text 中，进行 AI 模型推理时会从数据段.text 中读取各种指令。这样的实现方式表明模型的计算逻辑被神经网络编译器序列化，按照指令流的方式组织[32]，如图 5-1 所示即为一组简单的指令流。

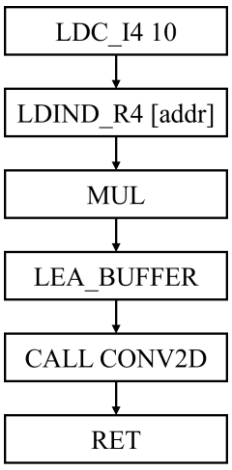


图 5-1 二维卷积指令流

由于算子的复杂程度不一，所以描述算子计算的指令流也各不相同，但不管简单还是复杂的指令流都可以解析为四个部分：数据加载（Load）、算子执行（Compute）、数据存储（Store）和流程控制（Call/Return）。

根据指令流的组成方式，我们便可以根据数据段.text 中的算子结构信息将上面四个部分映射到不同的线程中，但是由于指令流是序列化的，那么按照指令流组成部分划分的线程也会是按序执行，而这样不能充分利用 RTOS 的调度能力，故此本文提出从指令流中解析出 AI 模型算子结构的有向无环图（DAG），根据 DAG 图来划分线程就可以充分利用 RTOS 在多核下的线程调度，从指令流序列中解析算子结构的 DAG 图流程如图 5-2 所示。

中国知网 https://www.cnki.net

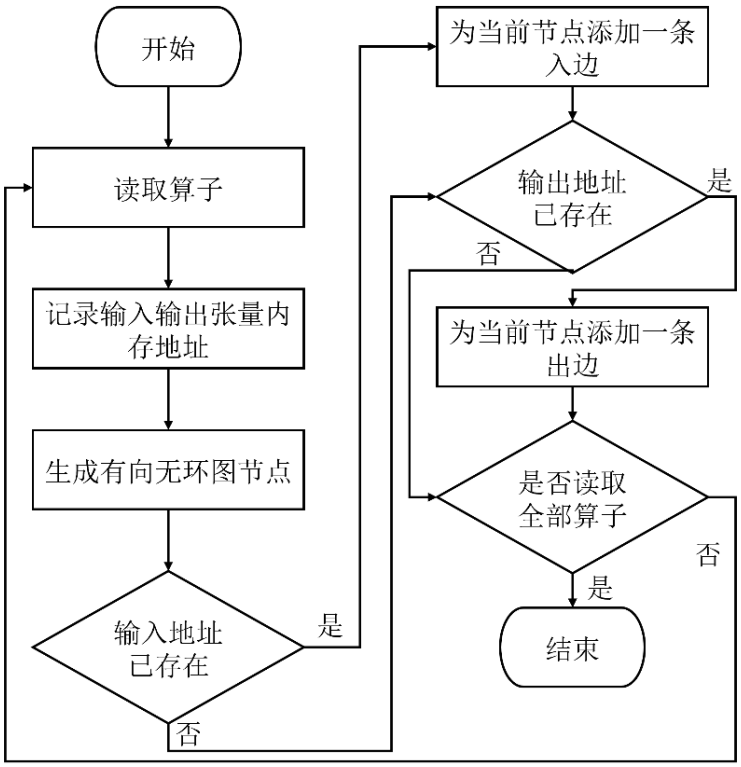


图 5-2 AI 模型算子结构 DAG 图生成

（1）读取数据段.text 中的算子结构，并记录下该算子输入输出张量的内存地址以及该算子涵盖的数据段.text 内容，最后生成一个有向无环图节点 Node；

（2）生成算子的 DAG 节点后，在记录的输出张量内存地址中查询该算子的输入张量内存地址，若地址已经存在则为该节点添加一条入边，连接到查询到的输出张量地址所在的 DAG 节点；

（3）在记录的输入张量内存地址中查询该算子的输出张量内存地址，若地址已经存在则为该节点添加一条出边，连接到查询到的输入张量地址所在的 DAG 节点；

（4）判断算子是否读取完毕，未读取完毕则重复（1）~（3）的工作，反之结束流程，DAG 生成完毕。

上述内容完成了从 AI 模型推理指令流中解析出算子结构 DAG 图的工作，本文框架中使用邻接表记录该 DAG 图，邻接表的节点定义如所示。

结构体 op_node 即为算子结构 DAG 图节点定义。其中 name 用于记录算子名称，比如（Conv2d_1、Relu_1、MaxPool_1 等）；dependencies 用于记录当前节点依赖的节点 ID，即 DAG 图中指向该节点的节点；dependents 用于记录依赖于当前节点的节点 ID，即 DAG 图中该节点指向的节点；start_addr 用于记录该算子节点

中国知网 https://www.cnki.net

在数据段.text 中的起始地址并用 text_size 记录大小，这两个变量供线程绑定时使用。

代码 5-1 算子结构 DAG 图节点定义代码

```
struct op_node {
    std::string name;
    std::vector<int> dependencies;
    std::vector<int> dependents;
    int start_addr;
    int text_size;
};
```

生成算子结构的 DAG 图之后就需要进行 RTOS 线程的映射，本文所使用的实时操作系统是珊瑚（aCoral）实时操作系统。针对 AI 任务的特殊性，aCoral 将 AI 任务的算子结构 DAG 图映射到一个线程集合，在这些 AI 任务线程中区别于传统线程的是会为每个线程绑定 AI 模型中数据段.text 中的一段内容，这段内容会包含一个或者多个算子操作的指令流，绑定完成后这若干个线程就相当于 DAG 图中的节点，操作系统便可以使用合适的调度策略完成 AI 推理任务。如图 5-3 所示是一个简单的 AI 任务映射为一个线程集合的示例。

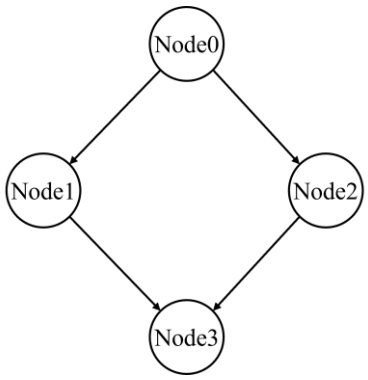


图 5-3 AI 任务 DAG 节点集合

在这个简单的 AI 任务中，算子结构 DAG 图仅有 4 个节点，实际情况可能更复杂，DAG 图中的箭头指向表示了计算节点之间的前驱后置关系，只有当前置节点计算完毕后后续的节点才能开始计算，在进行线程映射时应该考虑这种依赖关系，避免过多的线程切换产生的开销。对于图 5-3 中的 DAG 图，可以将其映射为如图 5-4（1）所示的线程集合，其中 thread0 包含了 Node0 和 Node1 的计算过程，

中国知网 https://www.cnki.net

thread1 包含了 Node2 和 Node3 的计算过程。如图 5-4（2）所示是另一种映射方式，使用四个线程 thread0、thread1、thread2 和 thread3 分别完成 Node0、Node1、Node2 和 Node3 的计算过程。

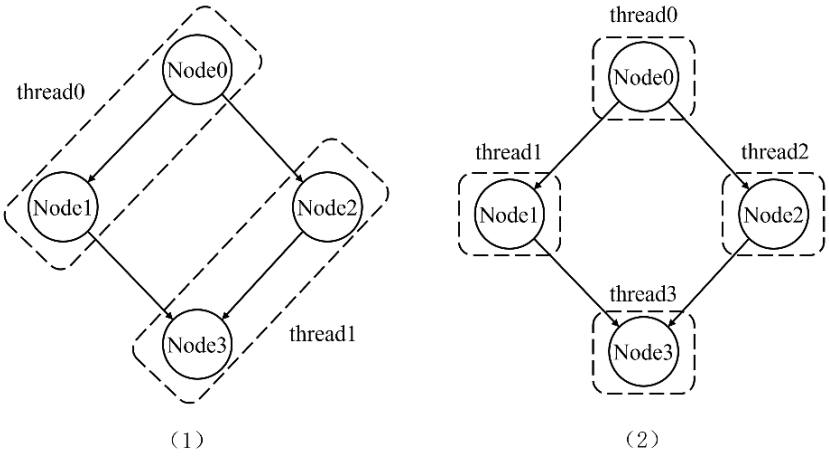


图 5-4 AI 任务 DAG 图线程映射方式

不同的映射方式带来的效果也不同，总体而言映射的粒度越小线程执行时的并行性就越高，AI 任务推理的效率也就更高，但同时线程繁多也会带来更多的线程切换开销[35]，用户进行开发时应该综合 AI 模型网络结构以及实际硬件环境进行评估来完成 AI 任务的线程映射。

5.2 AI 模型推理任务异构多核调度

在前面的章节中已经完成了 AI 任务的线程映射工作，并且在 4.4 小节中简略地提出了在异构核心下的任务负载划分。在 K210 硬件平台下拥有两颗 CPU 核心以及一颗神经网络处理单元 KPU，所以在完成 AI 任务的线程映射后需要将这些线程根据负载划分映射到各自的处理核心中执行。这些 AI 任务子线程的优先级与 AI 任务一致，整体上看 AI 任务只是作为 RTOS 中一个绑定有 AI 模型数据的持续任务，细分下的子线程之间拥有相同的优先级，它们的执行顺序依赖事件组标志位控制，并且由主核完成在异构多核下的线程调度过程[33]。

5.2.1 珊瑚（aCoral）实时操作系统事件组扩展

在珊瑚（aCoral）实时操作系统中已经实现了诸如信号量、互斥量、核间中断等线程通信方式，这些通信方式在简单的线程集合中进行线程执行顺序的控制和数据同步是较为方便简单的，但是在复杂的线程集合中这些通知方式就比较难以

中国知网 https://www.cnki.net

实现，对开发人员的代码水平有极高的要求。而由于 AI 模型推理任务在映射线程时是通过 AI 模型算子信息 DAG 图进行线程映射，这些线程之间存在复杂的前驱后继关系，线程调度时需要考虑这些依赖关系以保证线程按照正确的顺序执行。所以在本文框架中选择了另一种线程通信方式，事件组（Event Groups）。在嵌入式实时操作系统中，事件组同步机制允许多个线程等待或者发出一个或者多个事件标志（Bits）来完成线程的阻塞唤醒，这种通信方式天然的适合经过 AI 模型算子信息 DAG 图完成映射的线程集合，能够十分方便的将这些线程的前驱后继关系使用事件标志（Bits）来表示。

在 aCoral 中，事件组的数据结构定义如代码 5-2 中的结构体 EventGroupDef 所示，其中 uxEventBits 是一个 32 位的整型变量，每一位表示一个不同的事件标志；xTasksWaitingForBits 是一个任务队列，用于挂载等待事件的线程。

代码 5-2 事件组数据结构代码

```
typedef struct EventGroupDef
{       EventBits EventBits;
        acoral_list_t TasksWaitingForBits;
} EventGroup;
EventGroup* EventGroupCreate( void );
EventBits EventGroupSetBits(EventGroup* EventGroup, const EventBits BitsToSet);
EventBits EventGroupWaitBits(
        EventGroup* EventGroup,
        const EventBits BitsToWaitFor,
        const BaseType ClearOnExit,
        const BaseType WaitForAllBits,
        TickType TicksToWait );
EventBits EventGroupClearBits(EventGroup* EventGroup, const EventBits BitsToClear );
```

同时 aCoral 中为用户提供事件组相关接口，EventGroupCreate()用于创建事件组；EventGroupSetBits()用于设置事件组的事件标志；EventGroupWaitBits()用于等待事件标志；EventGroupClearBits()用于清除事件组中的事件标志。用户可调用这些接口完成线程集合中的流程控制与数据同步，保证任务的正确执行。事件组的内部流程如图 5-5 所示。

中国知网 https://www.cnki.net

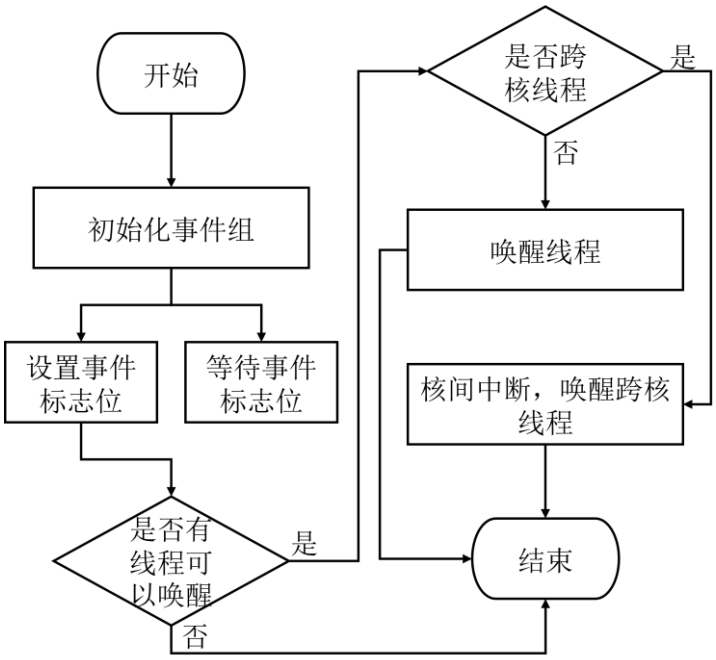


图 5-5 事件组内部执行流程

（1）用户使用事件组进行线程间数据同步时，首先调用 EventGroupCreate()函数，完成事件组的初始化，包括内存空间的申请和内部数据成员的初始化。

（2）在线程中如果需要等待某些事件发生则调用 EventGroupWaitBits()函数，事件组内部会将该线程挂载到自己的任务等待队列中，并阻塞该线程。

（3）当线程完成某些操作后需要通知等待该事件的其他线程时则调用 EventGroupSetBits()函数，该函数会根据线程传入的事件标志去唤醒自己的任务等待队列中等待该事件的线程。

（4）事件组进行线程唤醒的时候需要判断该待唤醒线程所在的核心，如果是在异核中的线程则需要借助核间中断通知核心唤醒任务。

5.2.2 AI 模型推理任务线程调度控制

本文框架中设计了一个主控线程负责 AI 任务线程的处理核心绑定过程，主控线程的处理流程如图 5-6 所示：

中国知网 https://www.cnki.net

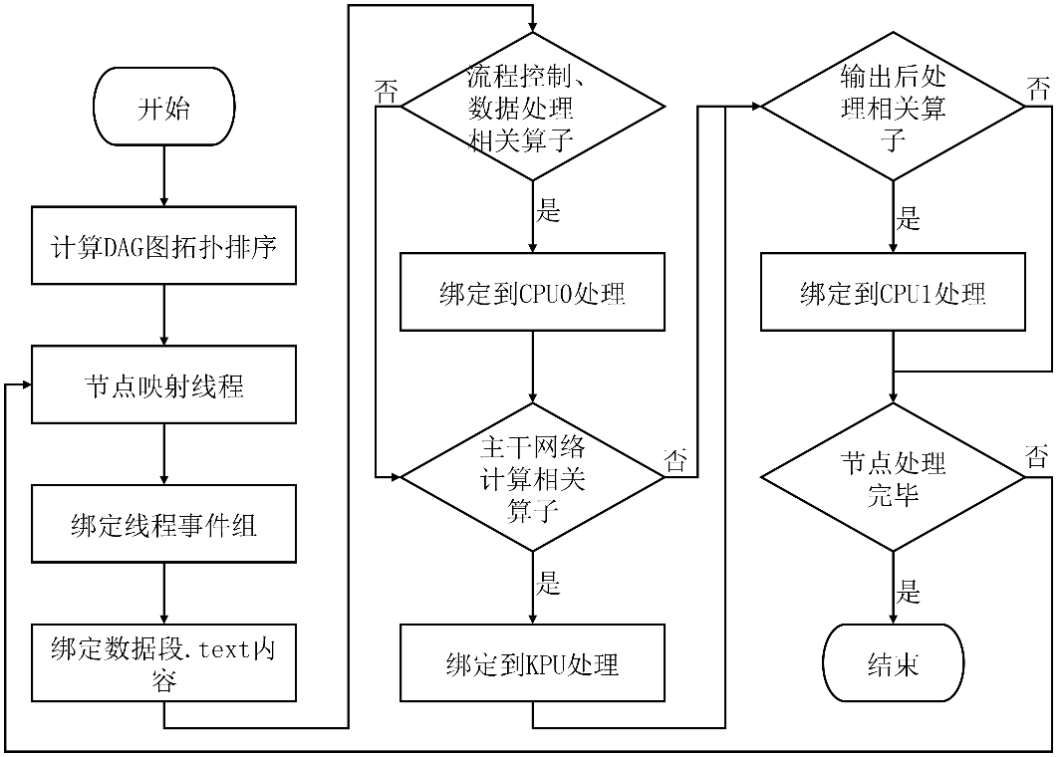


图 5-6 主控线程执行流程

（1）主控线程根据生成的 AI 任务 DAG 图计算节点的拓扑排序结果，DAG 图的拓扑排序代表了各个节点的依赖关系[34]，后续将根据排序结果确定线程的执行顺序；

（2）主控线程根据 AI 模型网络结构和实际硬件资源将节点划分为不同的线程供 RTOS 调度；

（3）根据 AI 任务 DAG 图的拓扑排序结果将一个或多个节点映射为线程，然后再为这些线程绑定事件组的标志位，事件组是一种 RTOS 中的任务通信方式，其作用与信号量类似，但一个事件组有多个标志位，意味着一个线程可以同时发出多个信号或者需要一次等待若干信号；

（4）根据 AI 任务 DAG 图节点中记录的 start_addr 和 text_size 成员为线程绑定 AI 模型数据段.text 中的内容；

（5）根据 AI 任务 DAG 图节点中算子的类型将线程绑定到不同处理单元进行处理；

（6）判断所有节点是否都已映射线程，若还有剩余节点则重复（1）~（5）的工作。

中国知网 https://www.cnki.net

5.3 AI 模型推理任务多核流水线加速方法

在传统的嵌入式 AI 推理框架中，AI 模型的推理流程是不透明的，并且不会做模型网络结构的拆解以进行并行计算，不能充分的利用嵌入式环境中多个核心以及神经网络处理单元的硬件资源[35]。

通过观察 AI 模型的算子结构以及本框架中使用的 Kmodel 文件结构发现，如果把在进行 AI 模型推理时的一系列算子计算过程看作一条指令流，那么一次 AI 模型的推理过程就与 CPU 中执行一条机器指令的过程非常类似，二者的对比如图 5-7 所示，AI 模型推理指令流的组成部分中流程控制（Call/Return）对应 CPU 指令生命周期的取指阶段，数据加载（Load）对应译码阶段，算子执行（Compute）对应执行阶段，数据存储（Store）对应访存阶段[36]。

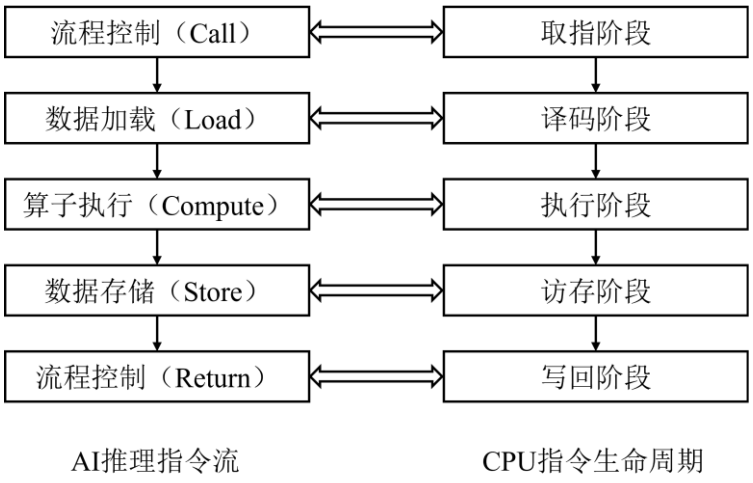


图 5-7 AI 推理指令流与 CPU 指令对比

再结合 AI 任务 DAG 图节点线程映射的方法，框架提出了在嵌入式多核环境下的 AI 推理任务流水线加速模型，该模型的描述如下：

（1）总任务数：m；计算核心数（即流水线级数）：n；AI 推理任务集合：D_1，D_2，…，D_m 每个 AI 推理任务 D_i 划分为 n 个子任务：T_1，T_2，…，T_n，分别由 n 个流水线阶段处理；单个流水线阶段处理时间：t_i；最长流水线阶段处理时间：$t_{max}=max(t_1,t_2,...,t_n)$。

（2）AI 推理任务串行执行时间（无流水线划分）：T_{serial}，该时间的计算方法如式(5-1)所示。

$$T_{serial} = m \cdot \sum_{i=1}^{n} t_i \tag{5-1}$$

中国知网 https://www.cnki.net

（3）任务集合在流水线模型下的执行时间：$T_{pipline}$，该时间的计算方式如式(5-2)所示。

$$T_{pipline} = (m + n - 1) \cdot t_{max} \tag{5-2}$$

（4）流水线模型的性能使用加速比进行描述：S_n，加速比越高代表流水线性能越好，加速比的计算方法如式(5-3)所示，当 $m>>n$ 时，加速比达到极限值。

$$S_n = \frac{T_{serial}}{T_{pipline}} = \frac{m \cdot \sum_{i=1}^{n} t_i}{(m+n-1)t_{max}} \tag{5-3}$$

（5）AI 推理任务流水线模型的性能还可以使用推理速度提升比进行描述：E_n，该数据的计算方法如式(5-4)所示。

$$E_n = \frac{T_{serial} - T_{pipline}}{T_{serial}} \tag{5-4}$$

（6）AI 推理任务流水线加速模型均衡性：B_n，该指标的计算方法如式(5-5)所示，当均衡性越接近 1 时表明流水线个计算阶段时间越均匀，加速比越接近极限值。

$$B_n = \frac{\sum_{i=1}^{n} t_i}{n \cdot t_{max}} \tag{5-5}$$

（6）在实际环境中影响 AI 推理任务流水线加速模型的加速比和推理速度提升比的因素主要有流水线级数（n）、总任务数（m）和 AI 推理任务流水线加速模型均衡性（B_n）。流水线级数（n）越大，总任务数（m）越多，均衡性（B_n）越接近 1，则流水线的性能越好。

在本文设计的框架中，AI 任务的流水线模型划分过程是在 AI 任务的主控线程中完成。在进行 AI 任务线程映射时，主控线程就会根据数据段.text 中的算子类型将 DAG 图中的节点映射到不同的线程，并通过静态绑定的方式将这些线程绑定到不同的处理单元进行执行，从而避免核间线程的迁移开销[37]。

本文在 K210 硬件平台中部署了图像识别任务，并且使用 AI 推理任务加速流水线模型完成了批量图像数据的推理过程。在 K210 中，硬件计算核心共有 3 个（CPU0、KPU 和 CPU1），框架将 AI 模型推理过程中的数据预处理过程，即图 5-7 中的流程控制（Call）和数据加载（Load）划分到 CPU0 执行，模型的主干网络计算过程，即图 5-7 中的算子执行（Compute）划分到 KPU 完成，最后由 CPU1 完成模型输出的后处理过程，即图 5-7 中的数据存储（Store）和流程控制（Return）。在 K210 中线程绑定到不同核心后的 AI 推理任务的流水线模型如图 5-8 所示。

中国知网 https://www.cnki.net

时间片	CPU0预处理	KPU推理	CPU1后处理
T1	预处理A	空闲	空闲
T2	预处理B	推理A	空闲
T3	预处理C	推理B	后处理A
T4	预处理D	推理C	后处理B

图 5-8 AI 推理任务流水线模型

图中描述了在 4 个时间片中 CPU0、KPU 和 CPU1 共同处理四个 AI 推理任务 A、B、C、D 的情况。假设每个处理过程都需要 1 个时间片完成并且忽视处理单元之间的数据传输延迟，那么可以预见在第 6 个时间片时这 4 个推理任务都将完成，而按照传统的单步执行（无流水线）推理方式则需要至少 12 个时间片才能完成 4 个任务的推理过程，理论上的推理效率提高了 50%。随着任务量的增加流水线模型带来的推理效率提升也将不断升高，在 K210 硬件平台下考虑 3 级流水线，当 AI 任务划分的流水线模型均衡性接近 1，在理想情况下的推理速度提升比为 66.6%，加速比为 3，如果 AI 推理过程能划分为更多的阶段并且有足够的处理单元进行运算，那么这一提升效率将更高[38]。

5.4 本章小结

本章主要介绍了 AI 任务的算子结构 DAG 图分析过程和线程映射方案以及在 aCoral 中实现的事件组同步机制，并且阐述了在 K210 硬件平台下的 AI 线程异构多核调度方案，最后针对嵌入式开发中的多处理器环境提出了 AI 推理任务流水线加速模型，并给出了在 3 级流水线下的推理性能提升表现。

中国知网 https://www.cnki.net

第六章 多模型 AI 推理框架的实现与测试

本章将对本文实现的资源受限嵌入式环境多模型 AI 推理框架的实现进行阐述，并结合珊瑚（aCoral）实时操作系统开发一个目标检测追踪系统，并在一些实际任务场景中完成测试。

6.1 资源受限嵌入式环境 AI 框架实现流程

前文章节介绍的 AI 模型虚拟内存管理器、AI 模型结构分析与负载划分和 AI 推理任务在异构多核下的调度策略以及流水线加速模型三者共同组成了本文研究的资源受限嵌入式环境多模型 AI 推理框架，框架的整体运行流程如图 6-1 所示。

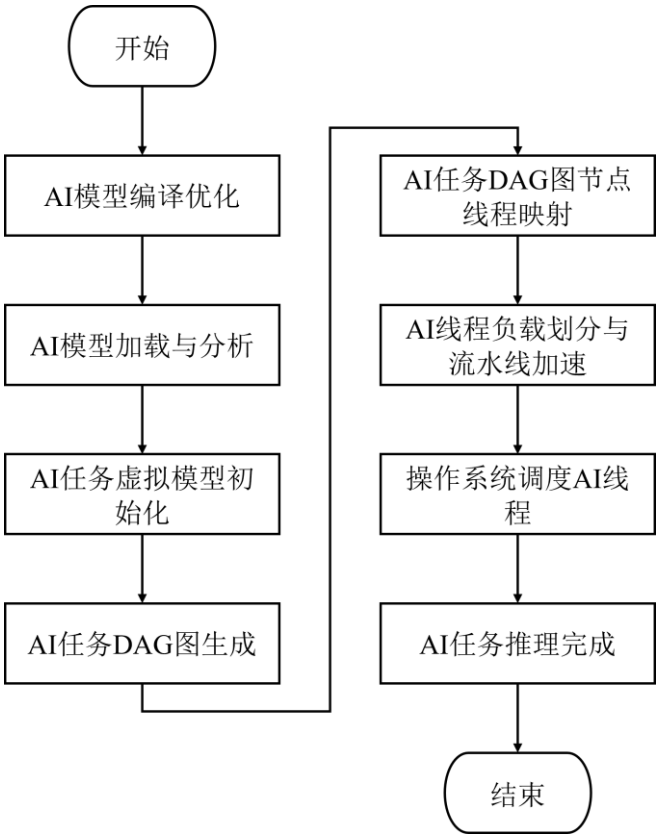


图 6-1 资源受限嵌入式环境多模型 AI 推理框架运行流程

（1）完成 AI 模型的编译优化。在 PC 端训练好的 AI 模型经过 nncase 神经网络编译器进行编译优化，生成在嵌入式端运行的 Kmodel 文件；

中国知网 https://www.cnki.net

（2）完成 AI 模型的加载分析。进行 AI 模型推理前，首先对模型进行加载，将其从外部存储器（NOR Flash）中读取到运行内存（SDRAM）中，并调用 kpu_process_kmodel()函数接口完成对模型结构的拆解分析；

（3）生成 AI 任务虚拟模型。根据 AI 模型的结构分析结果，并调用虚拟内存管理器的相关函数接口完成 AI 任务的虚拟模型初始化，后续推理工作都要依赖虚拟模型完成；

（4）生成 AI 任务 DAG 图。根据 AI 模型文件中的数据段解析出 AI 模型的算子结构，调用 ai_dag_init()接口以算子为节点生成 DAG 图，图中节点记录算子相关信息；

（5）完成 AI 任务 DAG 图节点的线程映射。根据各个节点所包含的算子信息将不同节点划分为不同的线程中，并结合珊瑚（aCoral）实时操作系统相关接口为线程绑定算子相关的指令内容；

（6）完成 AI 任务负载划分以及流水线推理加速。结合实际硬件平台和 AI 模型网络结构并通过指定线程结构体中的 core 参数完成各个处理单元的线程挂载，并仿照 CPU 中指令执行的流水线模型模拟 AI 模型的推理过程，提高系统的吞吐量；

（7）操作系统调度 AI 线程。在珊瑚（aCoral）实时操作系统中，AI 线程将与其他实时任务线程共同调度，并结合事件组（aCoral_eventgroup）和核间中断（clint_ipi_instance）等线程通信方式保证 AI 任务推理的正确性；

（8）AI 任务推理完成后处理。AI 模型完成推理后，针对推理结果进行后处理，并结合 K210 硬件条件完成图形化界面，向用户展示结果。

6.2 资源受限嵌入式环境多模型 AI 推理框架对比测试

本文所设计的资源受限嵌入式环境多模型 AI 推理框架是为了解决传统嵌入式 AI 推理框架在资源受限时模型体积过大无法加载、框架与嵌入式实时操作系统融合程度低和无法充分利用嵌入式端的硬件资源等问题而提出的，所以在完成资源受限嵌入式环境多模型 AI 推理框架的设计与实现后，本文通过实际的 AI 模型推理来对比传统框架与本框架之间的运行效果。

在模型的选取上本次实验选择了 Yolox 神经网络模型，Yolox 是一款由 Yolov3 和 Yolov5 改进优化而来的目标检测神经网络模型，Yolox 为用户在嵌入式端部署应用提供了轻量化模型 Yolox_Nano 模型，Yolox_Nano 模型针对嵌入式端部署进行了网络结构的裁剪以及参数量化等操作[39]。

中国知网 https://www.cnki.net

由于 K210 硬件条件的限制，仅有 6MB 运行内存供用户使用，本文使用 nncase 神经网络编译器对 Yolox_Nano 模型的 tflite 文件进行了编译优化得到 float 和 uint8 量化两种版本，它们各自的参数信息如表 6-1 所示[40]。

表 6-1 Yolox_Nano 两种版本参数信息

Model	Size(MB)	Params(M)	FLOSs(G)
Yolox_Nano_uint8.kmodel	1.01	0.91	1.08
Yolox_Nano_float32.kmodel	3.50	0.91	1.08

为了验证资源受限嵌入式环境多模型 AI 推理框架使用虚拟内存管理机制来辅助 AI 任务推理过程是否有效，本文设计了 Yolox_Nano 模型在两种框架下的对比实验。

首先选取 Yolox_Nano 模型的 float 版本部署在 K210 中，使其在 nncase 推理框架和本文设计的资源受限嵌入式环境多模型 AI 推理框架中对一张输入为 1x3x224x224 的带有三个人物的图片进行识别，识别结果对比如图 6-2 所示。



框架	模型大小（MB）	内存需求（MB）	识别效果
nncase推理框架	3.50	7.68	推理失败，内存不足
资源受限嵌入式环境多AI模型推理框架	3.50	7.68	推理成功，三个人物均成功识别

图 6-2 Yolox_Nano 模型的 Float 版本推理效果对比

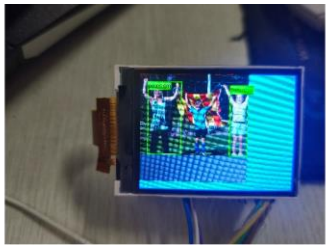
从实验结果我们可以看到，Yolox_Nano 模型的 float 版本在实际运行中的内存需求为 7.68MB，而这超出了 K210 仅 6MB 运行内存的限制，这导致了该模型在 nncase 推理框架下无法完成推理。而在本文使用虚拟内存管理机制来辅助 AI 任务推理的框架中模型则成功推理，并且正确识别出图像中的三个人物，而这在 Yolox_Nano 模型的 uint8 量化版本中因为精度问题并不能识别所有人物，这将在后续实验中得到验证。由此说明本文设计的资源受限嵌入式环境多模型 AI 推理框

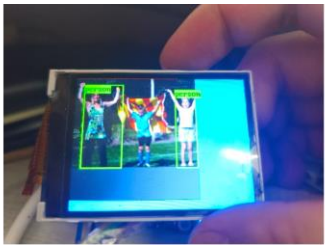
中国知网 https://www.cnki.net

架确实能解决在嵌入式环境中当物理内存不足以支撑 AI 模型推理时而推理失败的问题。

接着为了验证本文框架中使用的 AI 推理任务流水线模型确实能提高模型推理效率，我们设计了如下实验：

首先选取 Yolox_Nano 模型的 uint8 量化版本部署在 K210 中，使其分别在 nncase 推理框架和本文设计的资源受限嵌入式环境多模型 AI 推理框架中进行 10 次模型推理，观察它们各自的推理效果，如图 6-3 所示。





框架	推理总时长（S）	绝对推理时长（S）	识别效果
nncase推理框架	206.452s	20.342s	推理成功，仅识别出两个人物
资源受限嵌入式环境多AI模型推理框架	102.531s	21. 002s	推理成功，仅识别出两个人物

图 6-3 Yolox_Nano 模型的 uint8 量化版本推理效果对比

在两种框架下运行 Yolox_Nano 模型的 uint8 量化版本所需要的内存在 2MB 左右，均在 K210 的物理内存允许范围内。在对比实验中设计了两种框架下该模型对一张输入为 1x3x224x224 的带有三个人物的图片进行 10 次循环推理。在 nncase 推理框架下这 10 次推理的总时间为 206.452s，而在本文设计的框架中 10 次推理的时间仅为 102.531s，推理效率提高了约 49.5%，同时在两种框架下的推理效果也并无差别。这一结果充分证明了基于虚拟内存管理而实现的流水线加速模型确实能提升 AI 任务的推理效率。

为了说明在资源受限嵌入式环境中进行 AI 模型推理时，虚拟内存管理机制是否会对实时性产生一定影响，此处使用模型的绝对推理时长来比较两种框架的性能。如图 6-3 所示，在 nncase 推理框架中，每一帧图像的绝对推理时长在 20.342s 左右，而在本文设计的框架中，每一帧图像的绝对推理时长在 21.002s 左右。绝对推理耗时延迟了 1s 左右，耗时增加了原每一帧图像绝对推理时间的 5%左右，而

中国知网 https://www.cnki.net

10 次推理效率却提高了约 49.5%。由此说明，使用虚拟内存机制对 AI 模型推理过程进行管理，在进行长时间的推理任务中对实时性影响并无太大影响。

同时两种版本的模型在同一张图片上的不同识别效果也证明了在某些精度下量化模型无法达到识别要求，而要使用高精度模型就必须解决嵌入式环境中物理内存有限的问题，所以本文设计的框架中以虚拟内存管理来辅助 AI 模型推理则是一个不错的解决方案。关于本文框架与嵌入式实时操作系统高度融合以适应实际环境下多任务混合调度情况的说明将在下一小节中详细阐述。

6.3 基于目标检测追踪的实例验证

为了验证 AI 推理任务 DAG 线程确实能与嵌入式实时操作系统（RTOS）高度融合，能够在 RTOS 的调度机制下充分利用硬件资源，本文基于 K210 硬件平台设计了一套目标检测追踪系统。在此系统中将使用 Yolov2 模型进行目标检测并且通过 RTOS 调度云台控制线程对检测目标进行追踪。

6.3.1 系统设计与实现流程

本文使用串口烧写工具 Kflash 将珊瑚（aCoral）实时操作系统的二进制镜像文件与 Yolov2 模型 Kmodel 文件烧写到 K210 的外部存储器中，如图 6-4 所示。



图 6-4 系统烧写工具

中国知网 https://www.cnki.net

Kflash 通过串口 UART4 将珊瑚（aCoral）实时操作系统的二进制镜像文件烧录至外部存储器（NOR Flash）中 0x000000 地址中，Yolov2 模型的 Kmodel 文件烧录至 0xA00000 地址中。K210 上电以后，板级固件 Bootloader 将从 0x000000 地址读取珊瑚（aCoral）实时操作系统代码，操作系统的内核模块开始启动，后续的 AI 任务将从 0xA00000 地址中获取 Yolov2 模型的各种数据。

目标追踪系统依托于本文设计的资源受限嵌入式环境多模型 AI 推理框架与珊瑚（aCoral）实时操作系统实现。硬件上电后首先完成板级硬件初始化工作，包括系统时钟设置、串口硬件设置、外部存储器硬件设置、摄像头设置、显示器参数设置等工作；接着会将珊瑚（aCoral）操作系统二进制文件从外部存储器中读取到内存开始操作系统的初始化工作；操作系统内核模块初始化完毕后就会开始创建系统线程（init、daem、idle 线程），并且在 init 线程中创建用户任务线程，最后开始执行用户任务。珊瑚（aCoral）操作系统启动流程图如图 6-5 左侧所示。

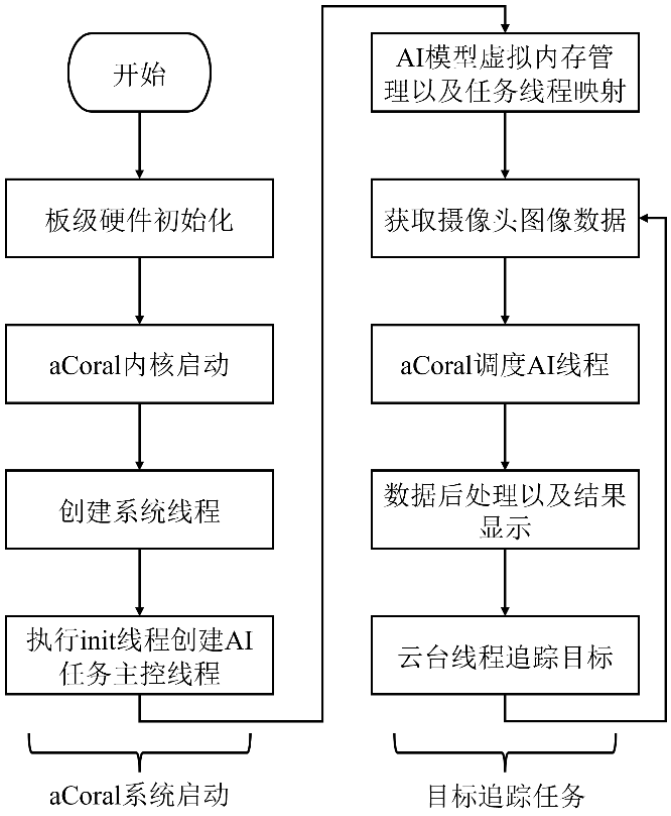


图 6-5 目标检测追踪系统运行流程

本小节设计的目标追踪系统将在珊瑚（aCoral）实时操作系统启动完毕后由 init 线程创建主控线程完成 AI 任务的线程以及追踪任务的创建，目标追踪系统执行流程如图 6-5 右侧所示。

中国知网 https://www.cnki.net

步骤 1，主控线程调用资源受限嵌入式环境多模型 AI 推理框架中的相关接口对 Yolov2 模型进行结构解析，完成模型的虚拟内存管理并通过模型的 DAG 图将模型映射到不同的线程。

步骤 2，利用摄像头获取图像，并对图像数据进行预处理，包括分辨率调整、RGB 转换等工作。

步骤 3，通过珊瑚（aCoral）实时操作系统的调度 AI 线程在不同处理单元之间进行计算推理。

步骤 4，推理结束后对模型的输出结果进行后处理，并展示到显示屏上。

步骤 5，根据 Yolov2 模型的识别结果，云台控制线程利用 PID 算法追踪识别到的物体。

步骤 6，循环执行步骤 1~5 的工作，除非出现异常，否则系统会一直执行。

从硬件启动到目标追踪任务不断执行的过程中，珊瑚（aCoral）实时操作系统会通过串口打印信息，如图 6-6 所示。系统启动完毕并完成 AI 任务线程以及云台线程创建后，便会开始调度线程，AI 线程与云台控制线程二者开始协作完成目标追踪任务。

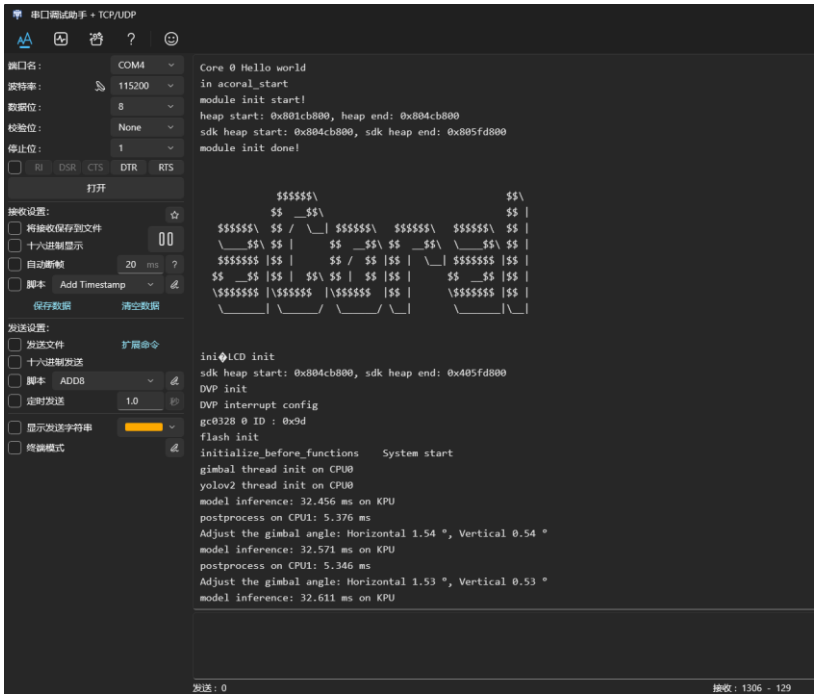


图 6-6 目标追踪系统运行串口信息

中国知网 https://www.cnki.net

6.3.2 K210 核心负载分析

为了观察在混合任务下各个核心的负载情况，验证本文设计的资源受限嵌入式环境多模型 AI 推理框架与珊瑚（aCoral）实时操作系统共同协作下确实能够充分利用硬件资源，本文设计了如表 6-2 和表 6-3 所示的任务集合 T，对比测试了使用 nncase 推理框架和资源受限嵌入式环境多模型 AI 推理框架时的系统负载情况。

表 6-2 nncase 推理框架下的任务集合 T

任务	符号	周期	优先级	处理核心	执行时间
持续任务 1	N1	\	7	CPU0	20ms
周期任务 1	P1	150ms	7	CPU1	10ms
云台任务	G1	\	6	CPU1	10ms
Yolov2 任务	T1	\	5	CPU0	40ms

表 6-3 资源受限嵌入式环境多模型 AI 推理框架下的任务集合 T

任务	符号	周期	优先级	处理器核心	执行时间
持续任务 1	N1	\	7	CPU0	20ms
周期任务 1	P1	150ms	7	CPU1	10ms
云台任务	G1	\	6	CPU1	10ms
Yolov2 任务 1	T1	\	5	CPU0	5ms
Yolov2 任务 2	T2		5	KPU	30ms
Yolov2 任务 3	T3		5	CPU1	5ms

任务集合 T 设置了一个持续任务 N1 和一个周期任务 P1 分别在 CPU0 和 CPU1 上执行，N1 只要 CPU0 空闲就会执行，P1 周期为 150ms，二者优先级为 7（数字越大优先级越低），执行时间分别为 20ms 和 10ms，云台控制任务 G1 在 CPU1 上执行，优先级为 6，执行时间为 5ms，Yolov2 任务在传统 nncase 推理框架中映射为一个线程 T1，在 CPU0 上执行，优先级为 5，执行时间在 40ms 左右，Yolov2 任务在本文设计的框架中被映射为三个线程 T1、T2、T3，分别在 CPU0、KPU 和 CPU1 上执行，优先级均为 5，执行时间分别在 5ms、30ms、5ms 左右。

在两种框架下任务集合 T 的运行时序如图 6-7 和图 6-8 所示，观察这两张时序图可以很容易的看出 300ms 的运行时间内，在 nncase 推理框架下目标追踪任务，即 Yolov2 任务 T1 和云台控制任务 G1 一共被调度了 6 次，持续任务 N1 一共被调度 3 次，周期任务 P1 一共调度了 2 次;而在使用资源受限嵌入式环境多模型 AI 推

中国知网 https://www.cnki.net

理框架的情况下 Yolov2 任务 T1、T2、T3 和云台控制任务 G1 一共被调度了 9 次，持续任务 N1 一共被调度 12 次，周期任务 P1 一共调度了 2 次。从这些任务的调度次数就能看出在本文实现的框架下硬件资源的利用率更高。

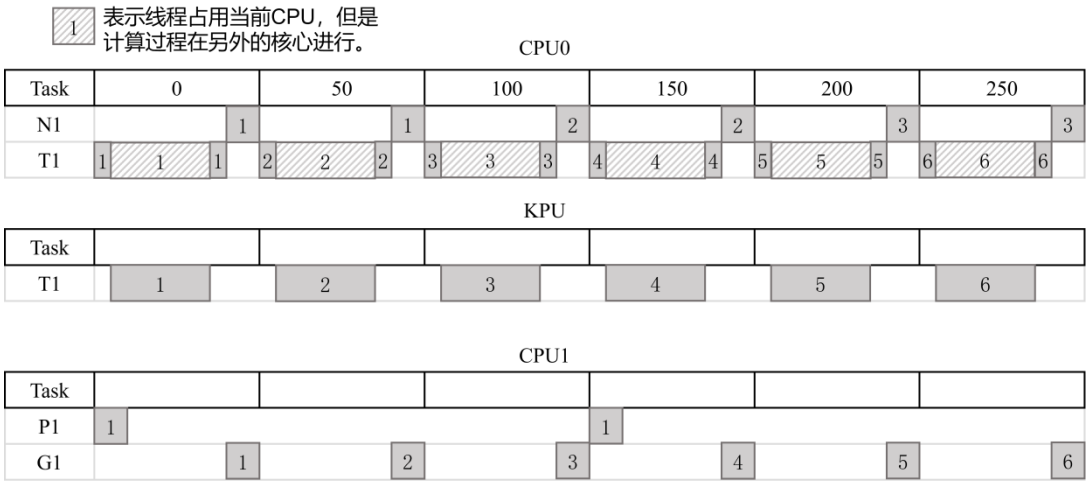


图 6-7 nncase 推理框架下的核心负载情况

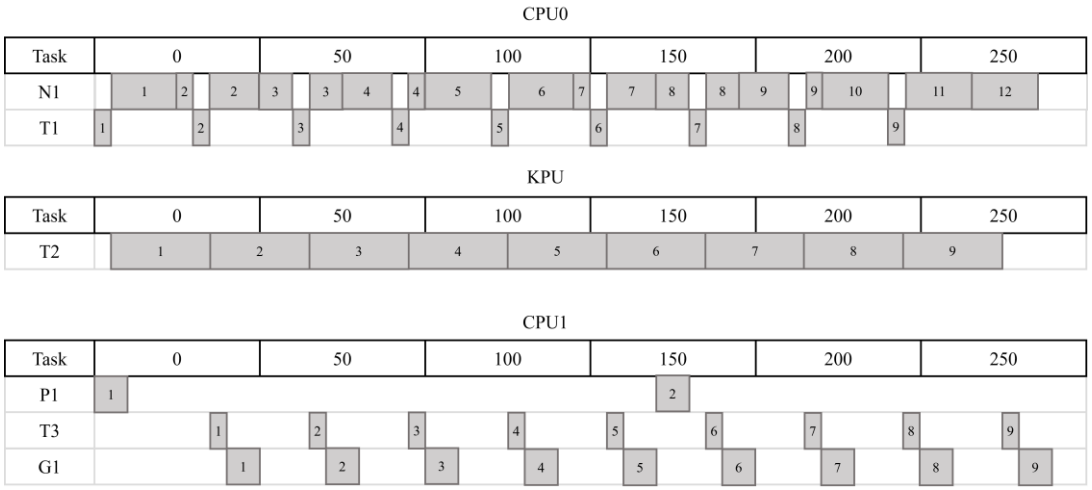


图 6-8 资源受限嵌入式环境多模型 AI 推理框架下的核心负载情况

根据两种框架的任务集合在 K210 硬件平台下的处理核心负载情况，我们可以计算出每个核心的利用率：U_c，用于对比两种框架在硬件资源利用率上的表现，核心利用率可根据式(6-1)计算得出，其中 *ti* 表示第 *i* 个任务执行一次所占用的处理核心时间，*Tp* 表示任务集合中所有周期任务的最小公倍周期，*m* 表示计算在 *m* 个最小公倍周期下的核心利用率。

中国知网 https://www.cnki.net

$$U_c = \frac{\sum_{i=1}^{n} t_i}{mT_p} \tag{6-1}$$

由以上数据可以得出，二者在各个核心的负载率数据对比如表 6-4 示，本文设计的资源受限嵌入式环境多模型 AI 推理框架在与珊瑚（aCoral）实时操作系统共同协作下充分的利用了嵌入式环境中的硬件资源，相对于传统的 AI 推理框架能够使各处理器核心的负载率得到有效提升。

表 6-4 两种框架下核心负载率对比

框架	CPU0 利用率	CPU1 利用率	KPU 利用率
nncase 推理框架	40%	26.6%	60%
资源受限嵌入式环境多模型 AI 推理框架	95%	51%	90%

使用资源受限嵌入式环境多模型 AI 推理框架与珊瑚（aCoral）实时操作系统将上述任务集合 T 部署在 K210 硬件中，上电后的运行结果如图 6-9 所示。可以发现实际运行效果与上文分析一致。

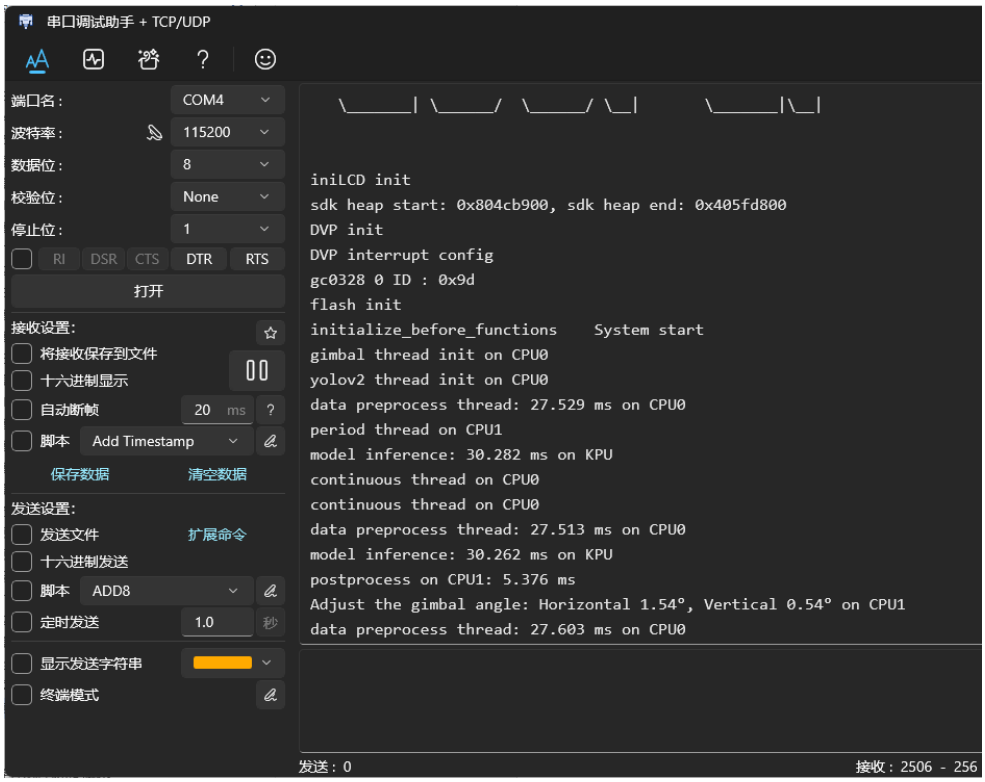


图 6-9 任务集 T 实际运行情况

中国知网 https://www.cnki.net

6.4 本章小结

本章介绍了资源受限嵌入式环境多模型 AI 推理框架的设计实现流程，并且使用 Yolox_Nano 模型在 nncase 推理框架下与本框架下进行了 AI 任务推理性能的对比测试实验，最后结合珊瑚（aCoral）实时操作系统完成了一个目标检测追踪系统，验证了资源受限嵌入式环境多模型 AI 推理框架结合实时操作系统以后在资源利用上良好的表现。

中国知网 https://www.cnki.net

第七章 全文总结与展望

7.1 全文总结

本文设计了一款资源受限嵌入式环境多模型 AI 推理框架，框架在无 MMU 环境中以软件的方式实现了虚拟内存管理器，对 AI 模型进行虚拟内存管理，解决了嵌入式环境中物理内存不足以支撑 AI 模型推理的问题，同时通过对 AI 模型的结构分析，将推理时的指令流转换为算子 DAG 图，使用 DAG 图完成 AI 任务的线程映射，并结合了珊瑚（aCoral）实时操作系统的多核任务调度机制进行线程调度，接着本文为了比较传统 AI 推理框架和资源受限嵌入式环境多模型 AI 推理框架在 AI 模型推理性能效果上的差异，使用 Yolox_Nano 模型进行了对比实验，实验结果证明了本文所设计框架在推理效率以及资源利用率上的优越性，最后设计了一套目标检测追踪系统，验证了在实际混合任务调度环境中，本文框架与实时操作系统的高度融合确实能充分利用硬件资源并保证任务的实时可控性。主要完成的工作如下：

（1）针对无 MMU 的嵌入式环境以软件的方式模拟 MMU 功能并实现了虚拟内存管理机制，并将虚拟内存管理机制融合到 AI 推理框架的模型加载推理之中，一定程度上解决了嵌入式开发中物理内存不足以支撑高精度 AI 模型推理的问题。

（2）对 AI 模型 Kmodel 文件进行结构的细致分析，使得 AI 模型加载推理过程透明可控，并通过 AI 模型推理时的指令流解析出 AI 任务 DAG 图，方便映射线程供嵌入式实时操作系统调度。

（3）根据 AI 任务 DAG 图和实际硬件情况，实现了 AI 任务的线程映射和异构多核下的调度策略，并提出了多核下 AI 推理任务的流水线加速模型。

（4）在 K210 硬件平台中部署珊瑚（aCoral）实时操作系统，基于 Yolox_Nano 模型完成了 nncase 推理框架和资源受限嵌入式环境多模型 AI 推理框架的对比实验，证明了本文设计框架 AI 任务推理的效率和资源利用率上的优越性。

（5）最后设计了目标检测追踪系统，结合珊瑚（aCoral）实时操作系统任务调度机制以及资源受限嵌入式环境多模型 AI 推理框架中提供的 AI 任务线程映射方式，验证了本文框架与 RTOS 的高度融合性，并且在混合任务调度下能保证任务的实时可控，并充分利用硬件资源。

中国知网 https://www.cnki.net

7.2 后续工作与展望

本文的工作仍然存在一些不足，在未来的开发中仍需要完成诸多工作，比如实现虚拟内存管理机制在整个嵌入式实时操作系统中的适配，使得本文实现的虚拟内存管理器不仅仅针对 AI 模型进行管理，对于用户的其他任务以及数据处理都能做到虚拟内存管理，实现线程的动态加载等功能；对于虚拟内存管理器模拟的 MMU 功能也要进一步完善，针对 cache 缓存实现相应的页面置换策略等，同时对于数据页面的换进换出也应该实现 DMA 方式控制的相应功能，减轻 CPU 负担；最后针对 AI 任务流水线推理加速模型实现更细粒度的任务负载划分，提供单次推理任务的多核心并行计算功能，降低 AI 任务的绝对推理时间，进一步提高 AI 推理任务的工作效率。

中国知网 https://www.cnki.net

参考文献

[1] 王琳博,白林亭,文鹏程.国产深度学习推理框架嵌入式适用性研究[J].航空计算技术,2023,53(3):121-125.

[2] Tensorflow:A system for large-scale machine learning[J]. M.Abadi;P.Barham;J.M.Chen;Z.F.Chen;A.Davis;J.Dean;M.Devin;S.Ghemawat;G.Irving;M.Isard;M.Kudlur;J.Levenberg;R.Monga;S.Moore;D.G.Murray;B.Steiner;P.Tucker;V.Vasudevan;P.Warden;M.Wicke;Y.Yu;X.Q.Zheng.Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation.2016.

[3] Embedded image processing on Raspberry Pi connected to the industrial control system[J]. Jozef Hrbček;Emília Bubeníková.Multidisciplinary Aspects of Production Engineering,2019.

[4] Android 移动应用集成华为 HMS Core 开放能力初探[J]. 邵国红.信息系统工程,2021(12).

[5] 陈班班. 基于深度学习的婴幼儿血管瘤风险评级算法设计与验证[D].华中科技大学,2023.DOI:10.27157/d.cnki.ghzku.2021.001374.

[6] ST. STM32Cube_AI 作为软件工具可优化在 STM32 上运行的人工神经网络[EB/OL]. (2023-3-15)[2024-2-2]. https://www.st.com/content/st_com/zh/campaigns/stm32cube-ai.html.

[7] Stmicroelectronics. Stm32fxx Datasheet[EB/OL]. [2023-12-30]. https://www.st.com/en/microcontrollers-microprocessors/stm32f103.html.

[8] Raspberry Pi Foundation. Rp2040 Datasheet[EB/OL]. [2023-12-30]. https://www.raspberrypi.com/.

[9] Texas Instruments. Msp430 Family Overview[EB/OL]. [2023-12-30]. https://www.ti.com/.

[10] Loongson Technology. Loongson 1C Processor Datasheet[EB/OL]. [2023-12-30]. http://www.loongson.cn/.

[11] Gigadevice. Gd32f103 Series Datasheet[EB/OL]. [2023-12-30]. https://www.gigadevice.com/.

[12] Xiaomi. Xiaomi Lantian Processor Datasheet[EB/OL]. [2023-12-30]. https://www.mi.com/.

[13] 一种基于树莓派的口罩自动检测装置[J]. 张丽艳;赵艺璇;李林.大连交通大学学报,2022(01).

[14] 基于深度学习的医疗垃圾分类的应用研究[D]. 牟钰鹏.大连交通大学,2022.

[15] 基于 ARM 的智能清扫车路面垃圾检测系统终端软件设计与实现[D].顾言.华中科技大学,2021.

[16] 华为 HMS 生态与应用开发实战[M]. 王希海.机械工业出版社.2020.

[17] 焦禹铭,吴凯,郭风祥等.基于专用卷积神经网络加速器的编译器设计与实现[J].计算机应用,2022,42(S1):208-214.

中国知网 https://www.cnki.net

[18] Abadi M, Barham P, Chen J, et al.TensorFlow: A system for large-scale machine learning[J].CoRR,2016,abs/1605.08695.

[19] 曾军,寇明阳,郑惜元等.TVM_T:基于 TVM 的高性能神经网络训练编译器[J].中国科学:信息科学,2023,53(12):2458-2471.

[20] Chen T, Li M, Li Y, et al.MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems[J].CoRR,2015,abs/1512.01274.

[21] 何小庆. 嵌入式操作系统风云录:历史演进与物联网未来[M]. 机械工业出版社, 2017.

[22] 何小庆. 多处理器电网调度自动化系统设计与实现 [D]. 北京;北京航空航天大学研究院, 1991.

[23] Hahm O, Tsiftes N, Petersen H, et al. Operating Systems for Low-End Devices in the Internet of Things: a Survey[J]. IEEE Internet of Things Journal, 2016, 3(5):720-734.

[24] 熊谱翔, 全召. RT-Thread Smart 微内核操作系统概述[J]. 单片机与嵌入式系统应用, 2021, 21(3):5.

[25] Silva M, Cerdeira D, Pinto S, et al. Operating Systems for Internet of Things Low-end Devices: Analysis and Benchmarking[J]. IEEE Internet of Things Journal, 2019, 6(6):10375-10383.

[26] Jim Cooling. Real-time Operating System Book1[J].Theory Lindentree Association, 2021.

[27] 黄一智, 李仁发. 从安全角度评述汽车操作系统[J]. 单片机与嵌入式系统应用, 2021, 21(3):6.

[28] 郭建川, 殷灿菊. 面向数控机床的可配置混合异构系统体系架构设计[J]. 单片机与嵌入式系统应用, 2022, 22(3):4.

[29] Barry R. Quickly add connectivity using FreeRTOS components Webinar Series/nxp.

[30] 韦开仁,李健,包晟临.多核实时操作系统的确定性调度设计[J].单片机与嵌入式系统应用,2020,20(12):7-9+14.

[31] 黄国睿,张平,魏广博.多核处理器的关键技术及其发展趋势[J].计算机工程与设计,2009,30(10):2414-2418.DOI:10.16208/j.issn1000-7024.2009.10.008.

[32] 朱扬烁,吕海玉,李奕晨等.多核处理器容错实时调度算法[J/OL].电子科技:1-9[2024-03-15].https://doi.org/10.16180/j.cnki.issn1007-7820.2025.01.010.

[33] 穆明.基于 K210 开发板的卷积神经网络教学设计[J/OL].中国教育技术装备:1-3[2024-03-15].http://kns.cnki.net/kcms/detail/11.4754.T.20230803.1545.002.html.

[34] 赵婉芳,陈莉莉.基于多核处理器的 RTOS 系统分析探究[J].软件,2023,44(02):123-125.

[35] 徐双国,刘云.基于核间中断实现的嵌入式多核系统通信方法[J].舰船电子工程,2021,41(05):58-61+95.

中国知网 https://www.cnki.net

[36] Li F, Bi R, Wang J, et al.VPSS: A DAG scheduling heuristic with improved response time bound[J].Journal of Systems Architecture,2024,148103084.

[37] Qamhieh M, George L, Midonnet S.Stretching algorithm for global scheduling of real-time DAG tasks[J].Real-Time Systems,2019,55(1):32-62.

[38] 林宇晗,严健,王侃侃等.支持缓存划分的全局 EDF 实时系统调度策略[J].东北大学学报(自然科学版),2021,42(12):1673-1680.

[39] 刘潇元,任钊婷,杨晨.基于 TINY YOLO2 神经网络视觉翻译棒[J].电脑知识与技术,2021,17(15):182-183+189.DOI:10.14004/j.cnki.ckt.2021.1507.

[40] Tolmacheva E A ,Ogurcov A D ,Dorrer G M .Puck tracking system for aerohockey game with YOLO2[J].Journal of Physics: Conference Series,2019,1399(3):033116-033116

中国知网 https://www.cnki.net